

Convolutional Neural Networks

CHEN PENG
SCHOOL OF ARTIFICIAL INTELLIGENCE
HEBEI UNIVERSITY OF TECHNOLOGY



Image Denoising and Convolution

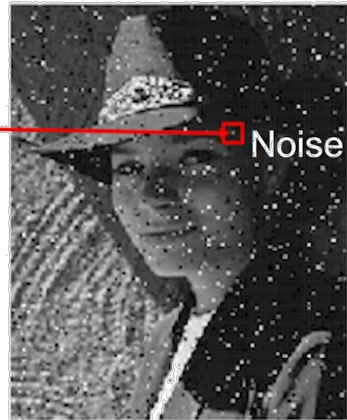
Image Denoising



Noisy Image

Image Denoising

64	76	69
71	253	75
68	74	78

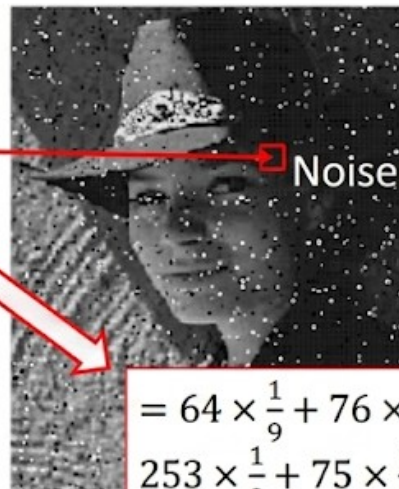


Noisy Image

Image Denoising

Before Denoising

64	76	69
71	253	75
68	74	78

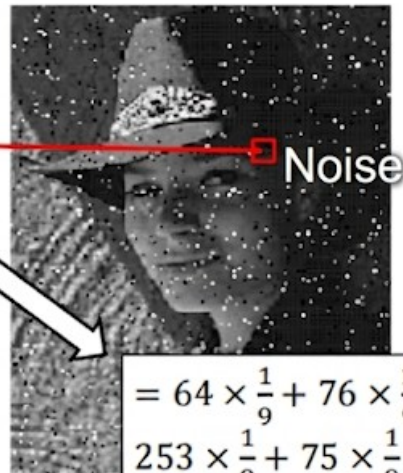


$$\begin{aligned} &= 64 \times \frac{1}{9} + 76 \times \frac{1}{9} + 69 \times \frac{1}{9} + 71 \times \frac{1}{9} + \\ &253 \times \frac{1}{9} + 75 \times \frac{1}{9} + 68 \times \frac{1}{9} + 74 \times \frac{1}{9} + \\ &78 \times \frac{1}{9} = 92 \end{aligned}$$

Image Denoising

Before Denoising

64	76	69
71	253	75
68	74	78

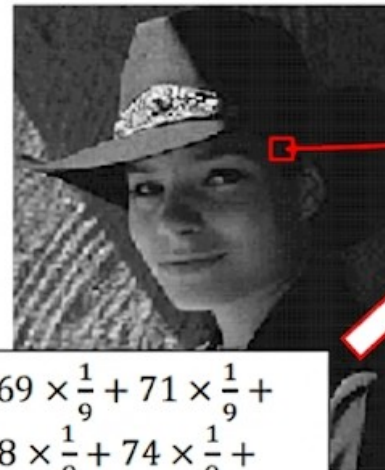


Noise

$$= 64 \times \frac{1}{9} + 76 \times \frac{1}{9} + 69 \times \frac{1}{9} + 71 \times \frac{1}{9} + 253 \times \frac{1}{9} + 75 \times \frac{1}{9} + 68 \times \frac{1}{9} + 74 \times \frac{1}{9} + 78 \times \frac{1}{9} = 92$$

After Denoising

64	76	69
71	92	75
68	74	78



Convolution Kernel

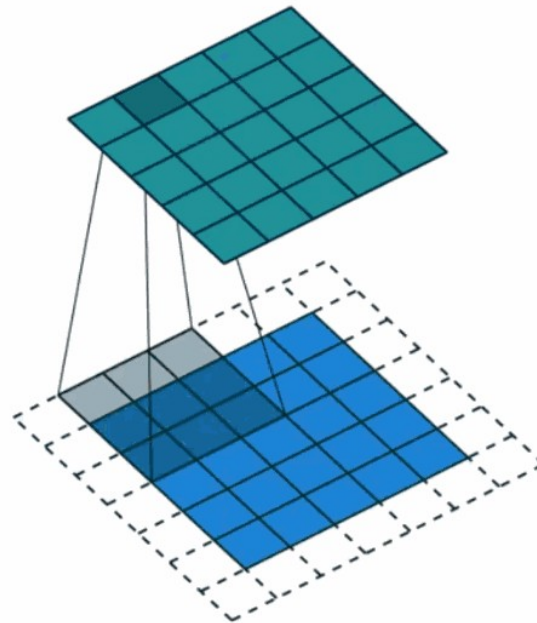
 $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

Convolution Kernel

Convolution Kernel

Convolved Image



Input Image

Definition of Convolution

Let F be the image, H be the convolution kernel, and the convolution of F and H be denoted as $R = F * H$.

$$R_{ij} = \sum_{u,v} H_{i-u,j-v} F_{u,v}$$

Definition of Convolution

Let F be the image, H be the convolution kernel, and the convolution of F and H be denoted as $R = F * H$.

$$R_{ij} = \sum_{u,v} H_{i-u,j-v} F_{u,v}$$

Convolving F with H to obtain R

Properties of Convolution

- Additivity: $\text{filter}(f_1 + f_2) = \text{filter}(f_1) + \text{filter}(f_2)$
- Translation Invariance: $\text{filter}(\text{shift}(f)) = \text{shift}(\text{filter}(f))$

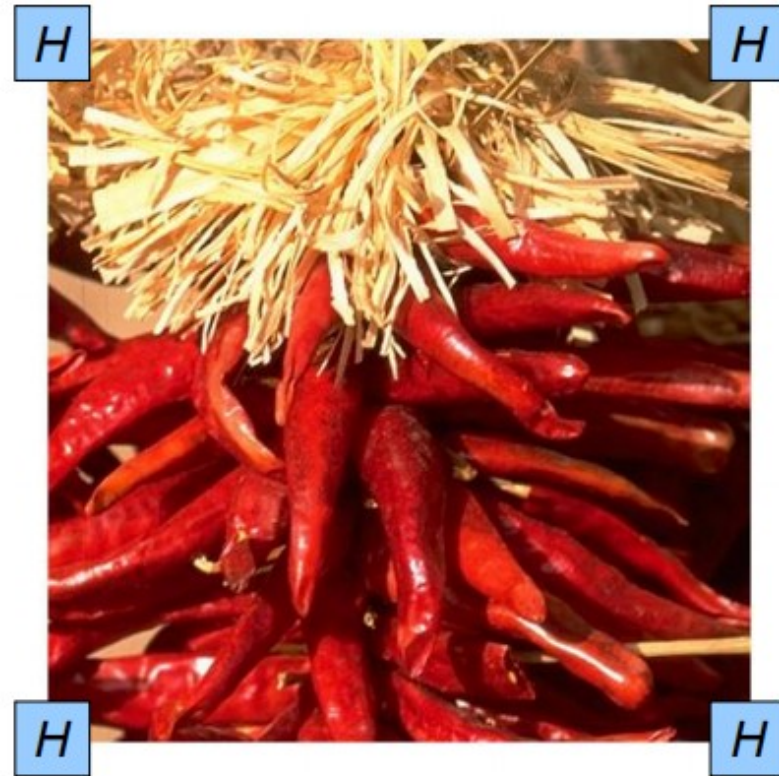
Properties of Convolution

- **Commutative Law:** $a * b = b * a$
- **Associative Law:** $a * (b * c) = (a * b) * c$
- **Distributive Law:** $a * (b + c) = (a * b) + (a * c)$
- **Scalar:** $ka * b = a * kb = k(a * b)$

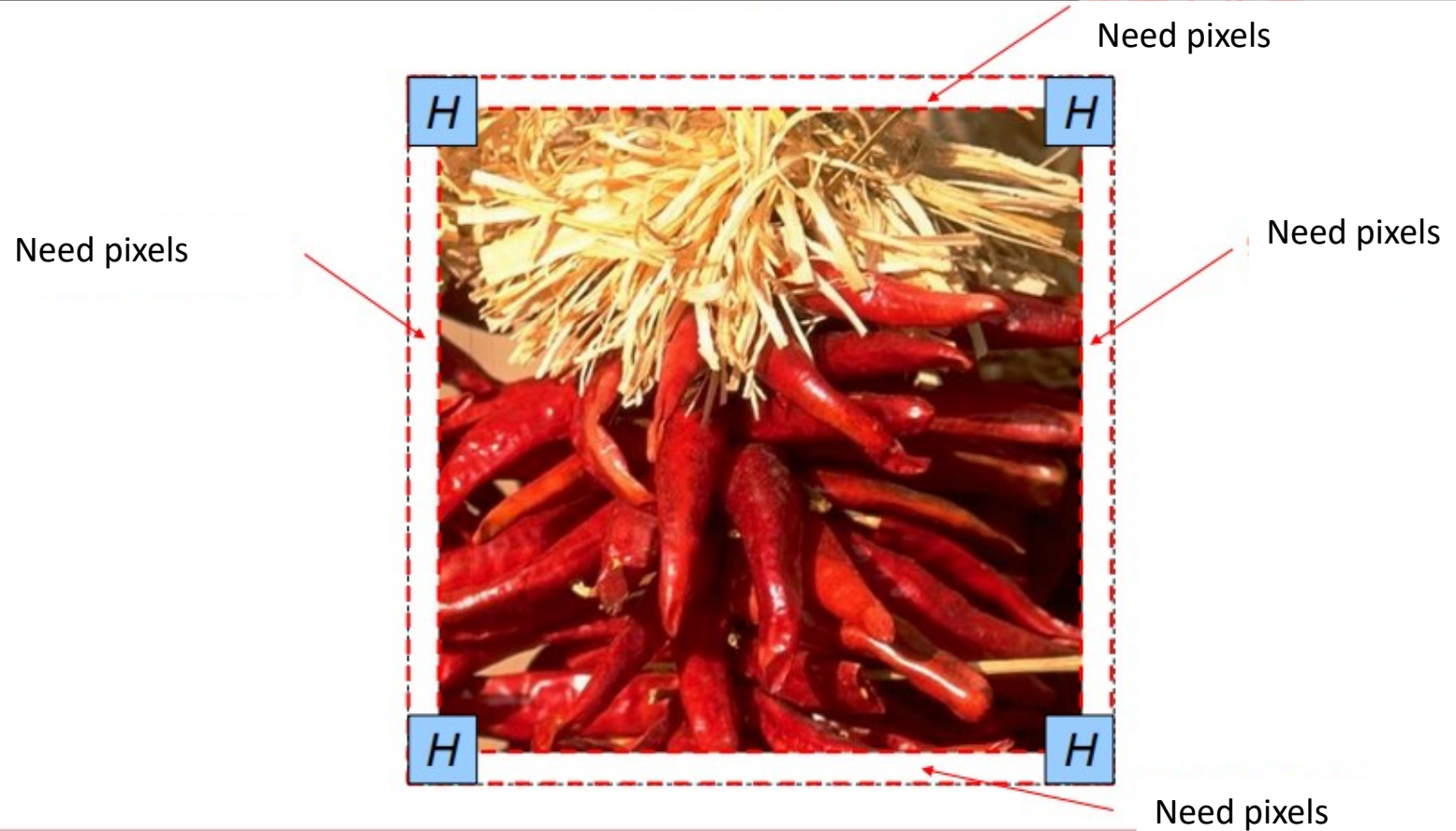
Summary

In this section, we learned a fundamental image operation, convolution. We hope you can understand what convolution is, master the key properties of convolution, and know how to use convolution kernels to convolve images.

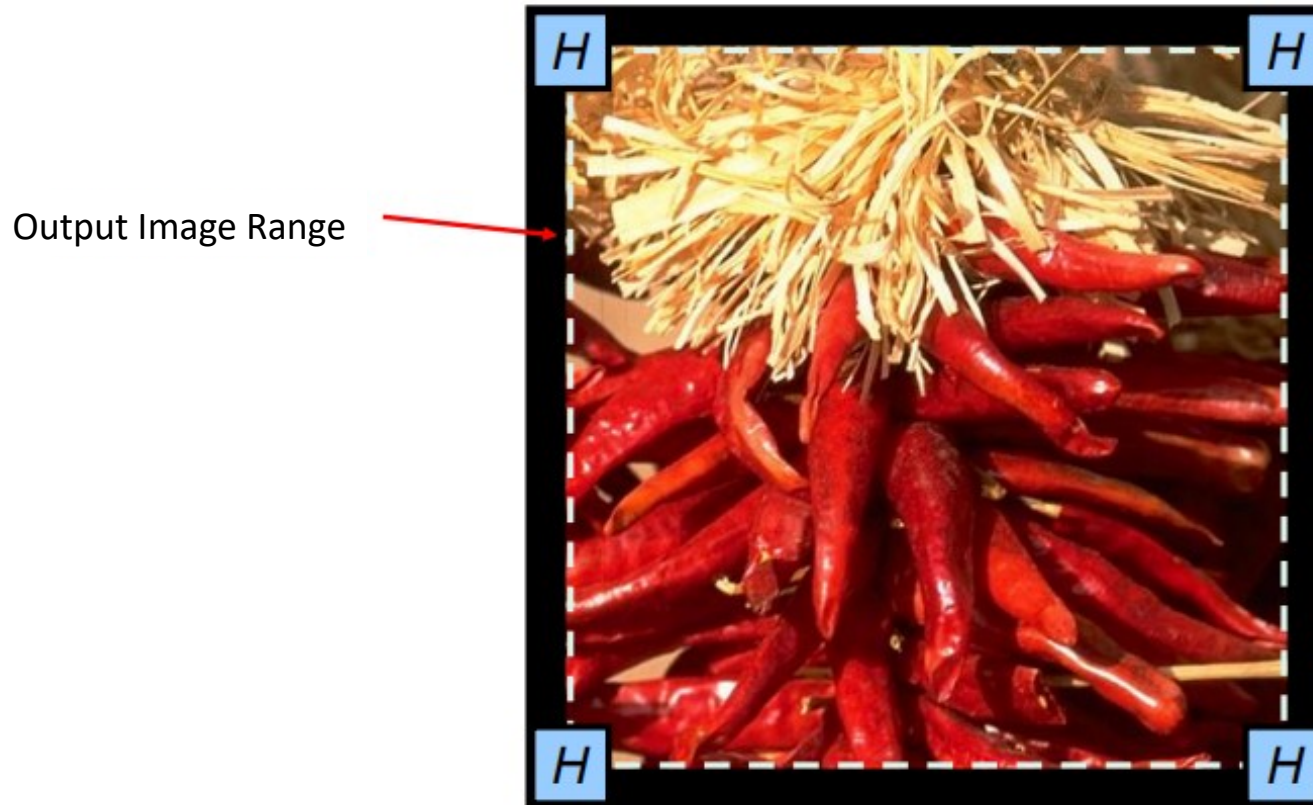
Boundary Padding



Boundary Padding



Boundary Padding



Boundary Padding



Stretching



Mirror

Summary

- The convolved image is smaller than the input. Through boundary padding, we can maintain the same image size before and after convolution;
- One of the most commonly used boundary padding methods is constant padding.

Convolution Example



Original Image

0	0	0
0	1	0
0	0	0

Convolution Example



Original Image

0	0	0
0	0	1
0	0	0

Convolution Example



Original Image

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

Convolution Example



Original Image

0	0	0
0	2	0
0	0	0

 -

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

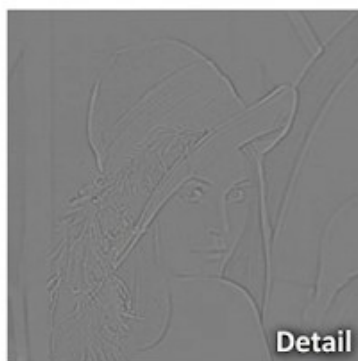
Sharpening



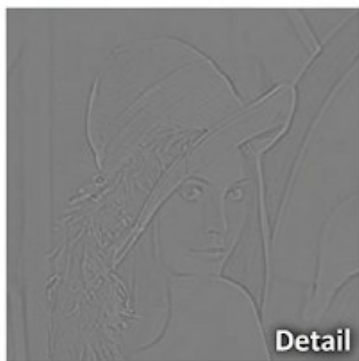
-



=



+



=



Sharpening

$$\begin{array}{c} \text{Original} \end{array} * \begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} + \begin{array}{c} \text{Original} \end{array} * \begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} - \begin{array}{c} \text{Original} \end{array} * \frac{1}{9} \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array}$$

$$= \begin{array}{c} \text{Original} \end{array} * \left(\begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 2 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} - \frac{1}{9} \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array} \right) = \begin{array}{c} \text{Sharpened} \end{array}$$

Summary

Convolution is a fundamental and important image operation in image processing. It can achieve:

- Translation
- Smoothing
- Sharpening

.....

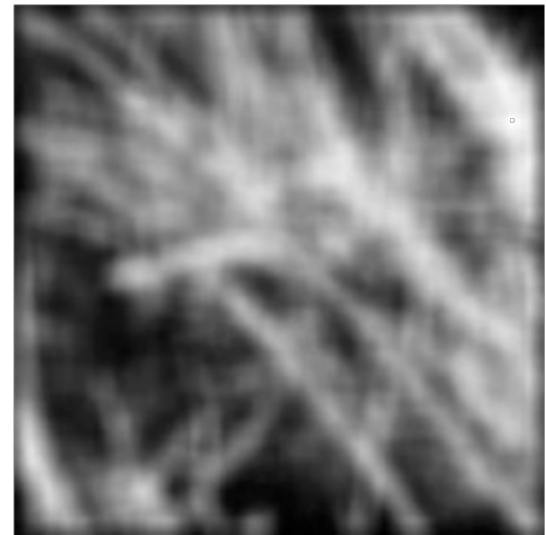
Gaussian Convolution Kernel

Problems with Average Convolution Kernel

Solution: Assign weights based on the distance between neighboring pixels and the center



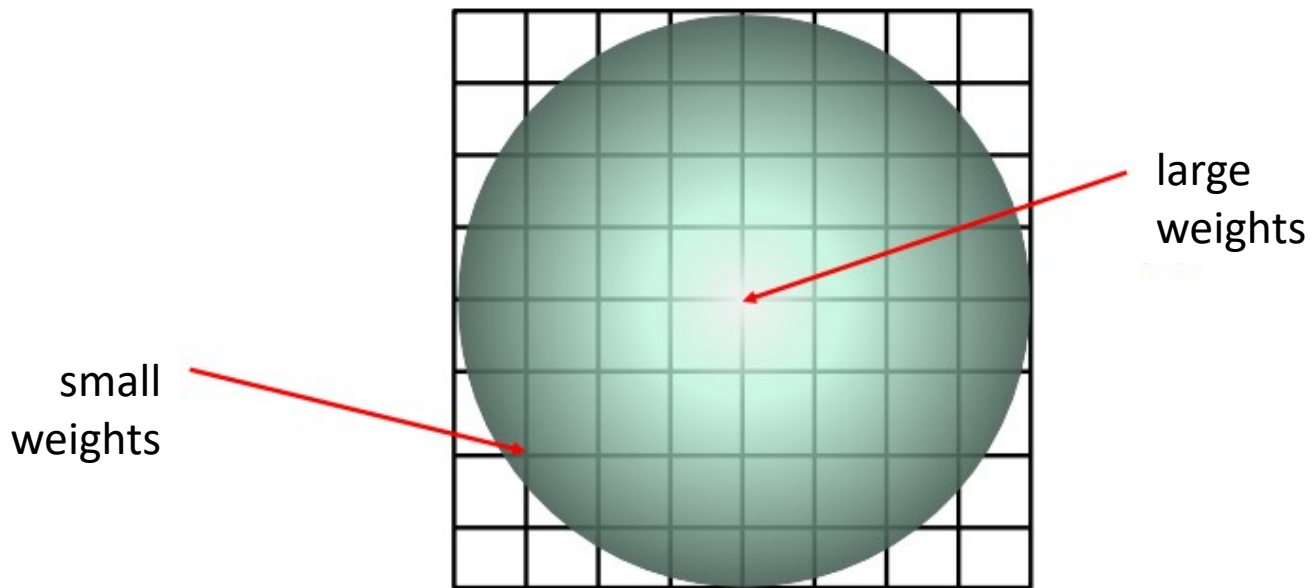
1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9



The convolved image produces horizontal and vertical stripe patterns — Ringing!

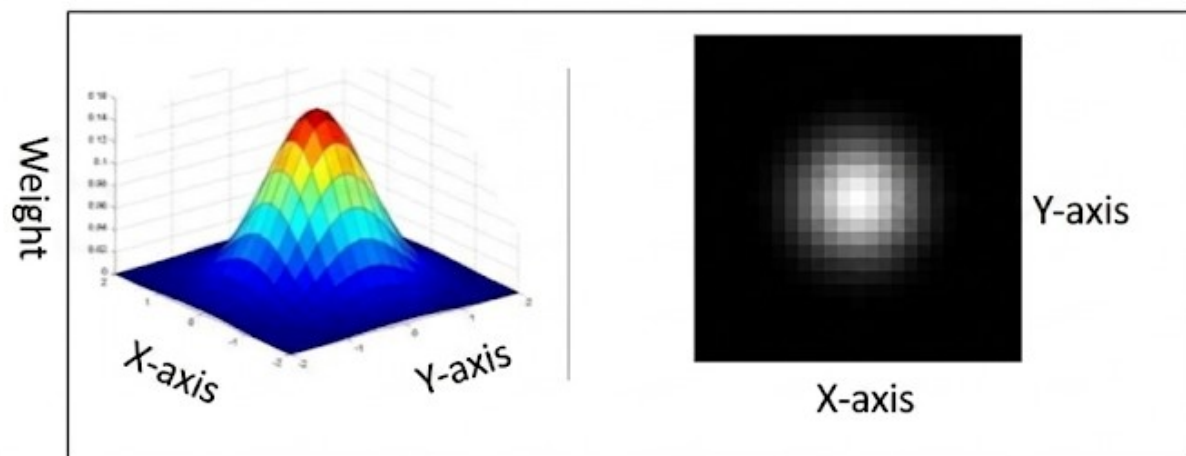
Problems with Average Convolution Kernel

Solution: Assign weights based on the distance between neighboring pixels and the center



Gaussian Convolution Kernel

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$



Visualization of Gaussian Kernel Weights

Gaussian Convolution Kernel

0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

Standard Deviation: $\sigma=1$

Gaussian Convolution Kernel

Generation Steps:

1. Determine the size of the kernel, for example 5×5
2. Set the standard deviation of the Gaussian function, for example $\sigma=1$

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

3. Calculate the weight values for each position of the kernel
4. Normalize the weight values

A 5×5 Gaussian convolution kernel

0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

Standard deviation: $\sigma=1$

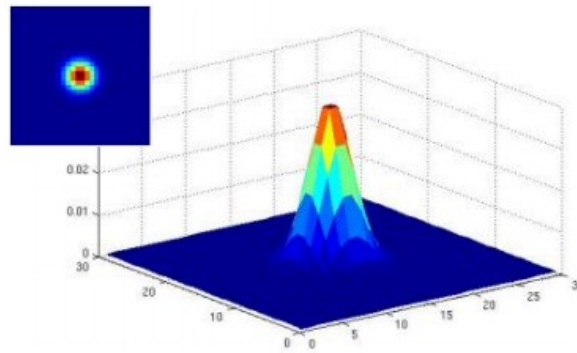
Gaussian Convolution Kernel

How to set the following parameters:

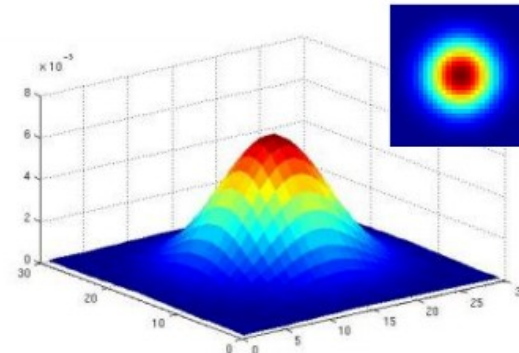
1. Kernel size
2. Standard deviation of the Gaussian function

Variance Variation

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$



30 x 30 $\sigma = 2$

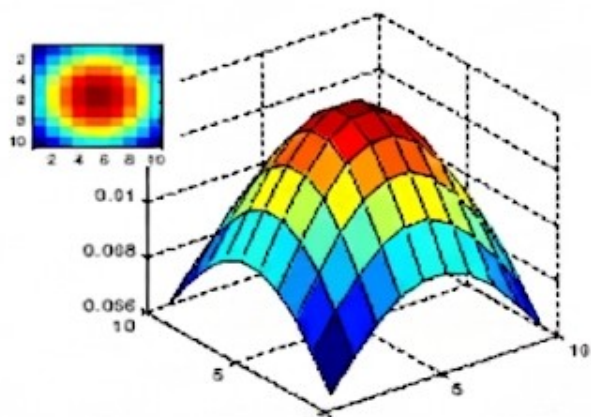


30 x 30 $\sigma = 5$

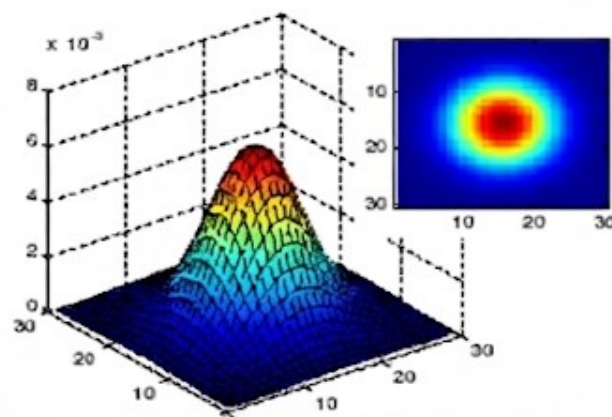
Larger variance $\rightarrow$ more obvious smoothing effect

Window Width Variation

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$



Gaussian Standard Deviation $\sigma = 5$
Template Size 10 x 10



Gaussian Standard Deviation $\sigma = 5$
Template Size 30 x 30

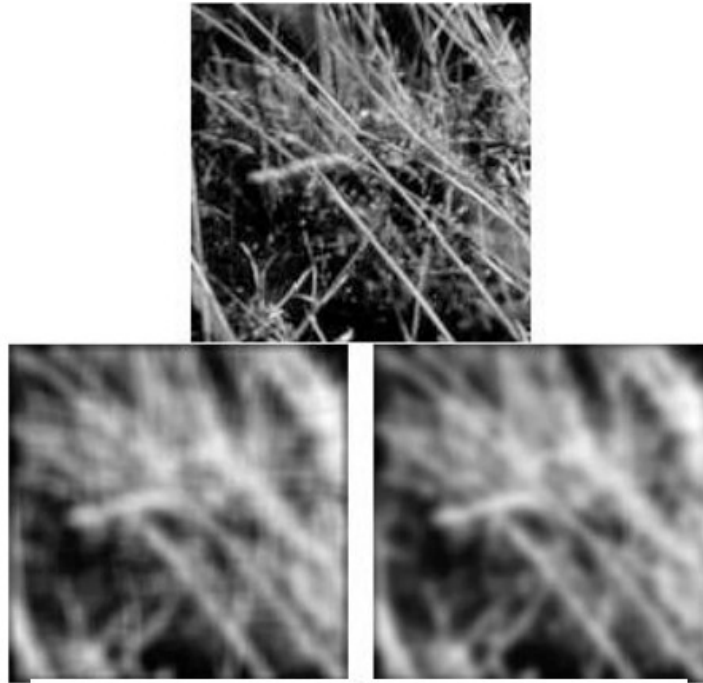
The larger the template size, the stronger the smoothing effect

Convolution Kernel Parameters

- Large-variance or large-size convolution kernels have stronger smoothing capabilities;
- Small-variance or small-size convolution kernels have weaker smoothing capabilities;
- Rule of Thumb: Set the half-window width of the convolution kernel to 3σ , for a final mask size of $2 \times 3\sigma + 1$.

Example: Standard deviation set to 1, convolution mask width = $2 \times 3 \times 1 + 1 = 7$.

Gaussian Convolution Kernel vs. Average Convolution Kernel



Average Kernel
Convolution Result

Gaussian Kernel
Convolution Result

Gaussian Convolution Kernel

- Remove “high-frequency” components in the image (low-pass filter)
- Convolution of two Gaussian convolution kernels results in a Gaussian convolution kernel
 - Consecutive convolution with multiple small-variance convolution kernels achieves the same result as with a large-variance convolution kernel
 - Two convolutions with a Gaussian kernel with standard deviation σ are equivalent to one convolution with a Gaussian kernel with standard $\sigma\sqrt{2}$
- Separable
 - Can be decomposed into the product of two 1D Gaussians

Separability Example

Gaussian Kernel
Convolution with Image

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} * \begin{bmatrix} 2 & 3 & 3 \\ 3 & 5 & 5 \\ 4 & 4 & 6 \end{bmatrix} = 65$$

$$\begin{aligned}
 &= 2 + 6 + 3 + \\
 &\quad 6 + 20 + 10 + \\
 &\quad 4 + 8 + 6 \\
 &= 65
 \end{aligned}$$

Decomposed into two
1D Gaussian Kernels

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

1D Kernel Convolution

$$\begin{bmatrix} 1 & 2 & 1 \end{bmatrix} * \begin{bmatrix} 2 & 3 & 3 \\ 3 & 5 & 5 \\ 4 & 4 & 6 \end{bmatrix} = \begin{bmatrix} & 11 & \\ & 18 & \\ & 18 & \end{bmatrix}$$

1D Kernel Convolution

$$\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \begin{bmatrix} & 11 & \\ & 18 & \\ & 18 & \end{bmatrix} = \begin{bmatrix} & & \\ & 65 & \\ & & \end{bmatrix}$$

Convolution Operation Computational Complexity

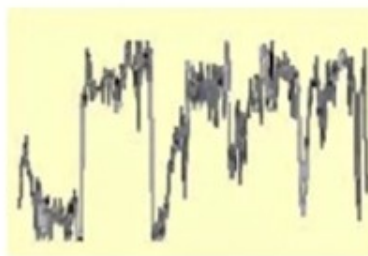
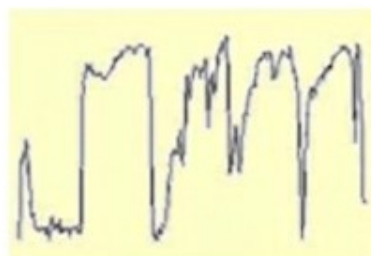
1. What is the computational complexity of convolving an $n \times n$ image with an $m \times m$ kernel?
2. What if the kernel is separable?

Summary

In this section, we have learned about a new type of convolution kernel, the Gaussian kernel, which can effectively suppress noise and achieve image smoothing. At the same time, we have also introduced the stacking and decomposition of Gaussian convolution kernels, both of which can be used to reduce the computational complexity of convolution.

Image Noise

Gaussian Noise

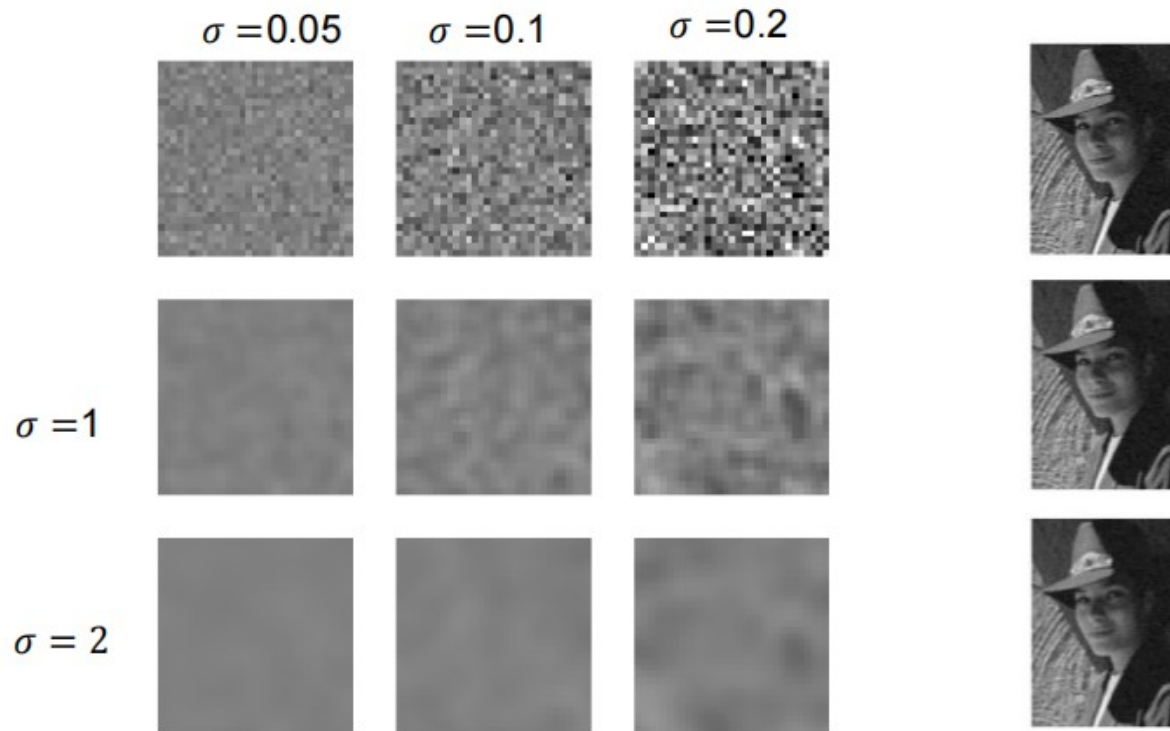


Ideal image Random noise
 $\hat{f}(x, y) = f(x, y) + \eta(x, y)$

Noise distribution
 $\eta(x, y) \sim \mathcal{N}(\mu, \sigma)$

Typically
 $\mu = 0, \sigma$ is very small.

Reducing Gaussian Noise



Noise



Original



Salt & Pepper Noise



Impulse Noise



Gaussian Noise

- **Salt & Pepper Noise:** Black and white pixels appear randomly
- **Impulse Noise:** White pixels appear randomly
- **Gaussian Noise:** Noise intensity variations follow a Gaussian distribution (normal distribution)

Salt-and-pepper Noise

3x3



5x5



7x7



Gaussian Kernel Denoising Result

Median Filter

Image

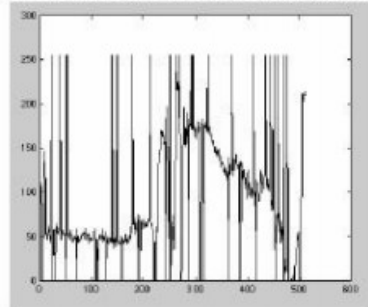
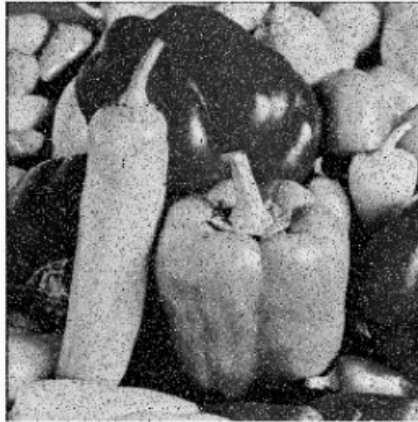
26	23	27	18	21
29	10	15	20	30
31	23	90	27	32
19	33	31	30	27
28	29	27	25	31



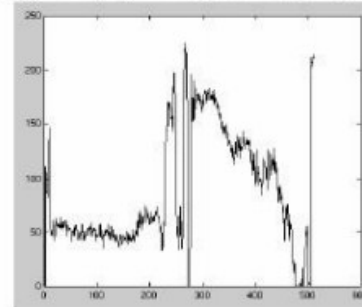
Sort

Median Filtering

Salt-and-pepper Noise



After Median Filtering



Summary

In this section, we learned three types of noise: salt-and-pepper noise, impulse noise, and Gaussian noise. For the first two, median filtering is recommended; for Gaussian noise, Gaussian convolution kernel can be used for denoising.

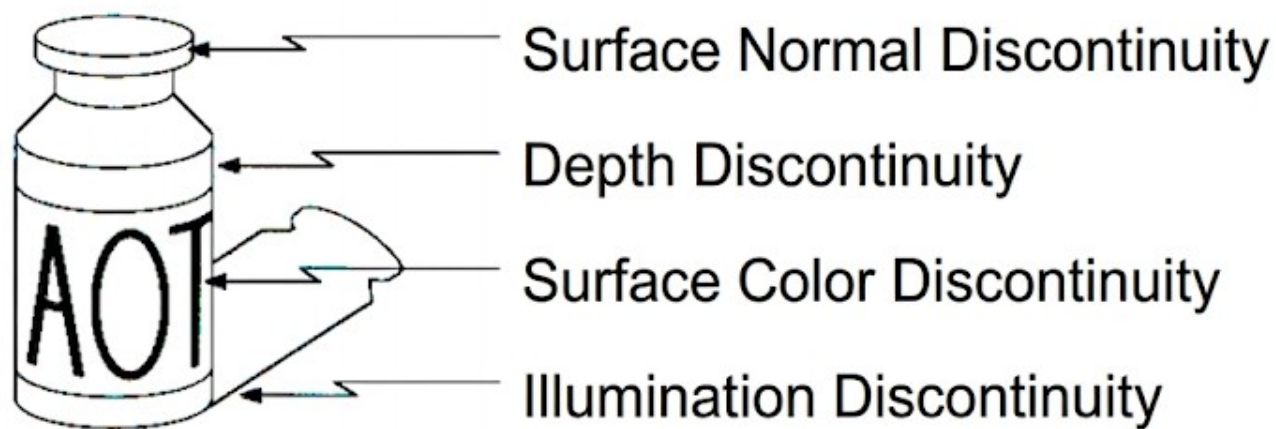
Edge Detection

Edge Extraction

- **Edge:** Points in an image with obvious and sharp brightness changes
- **Why study edges?**
 - Encode semantic and shape information in the image
 - Compared to pixel representation, edge representation is clearly more compact



Types of Edges



Edge Detection

Points in an image where brightness changes clearly and rapidly

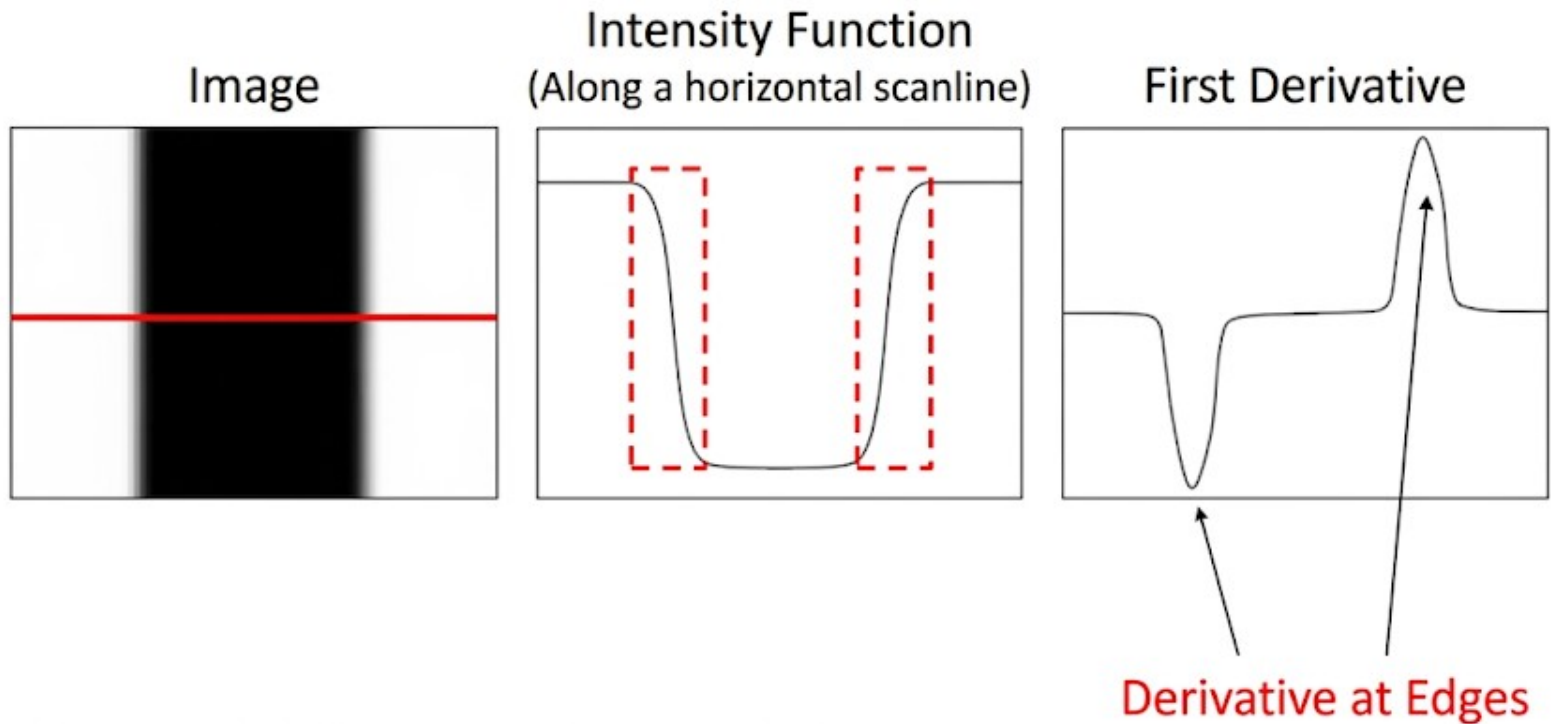


Image Derivative

The partial derivative of a 2D function $f(x,y)$ is:

$$\frac{\partial f(x,y)}{\partial x} = \lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon, y) - f(x, y)}{\varepsilon}$$

Image derivative formula:

$$\frac{\partial f(x,y)}{\partial x} \approx \frac{f(x + 1, y) - f(x, y)}{1}$$

$$\frac{\partial f(x,y)}{\partial x}$$

-1	1
----	---

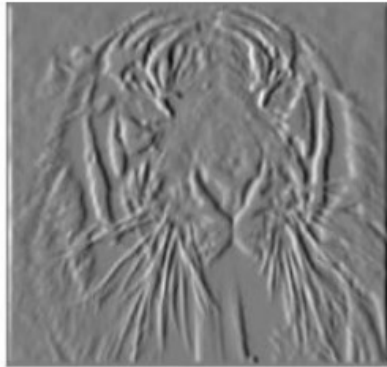
$$\frac{\partial f(x,y)}{\partial y}$$

-1
1

Image Partial Derivative

$$\frac{\partial f(x, y)}{\partial x}$$

-1	1
----	---

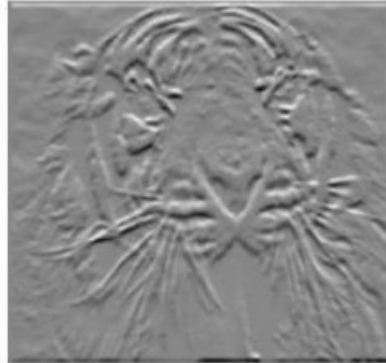


$$\frac{\partial f(x, y)}{\partial y}$$

-1
1

 or

1
-1

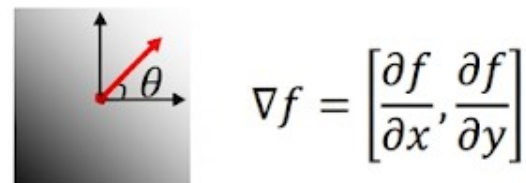
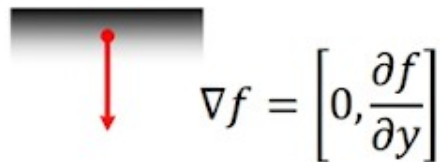
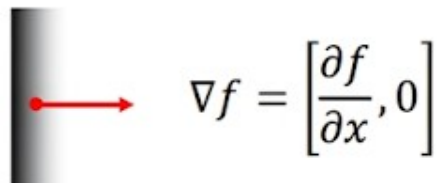


Source: K. Grauman

Image Gradient

Image Gradient: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$

Gradient points in the direction of the fastest gray-level change



Gradient Direction:

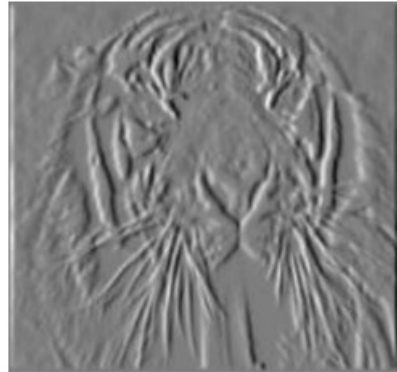
$$\theta = \tan^{-1} \left(\frac{\frac{\partial f}{\partial y}}{\frac{\partial f}{\partial x}} \right)$$

Relationship between gradient direction and edge direction?

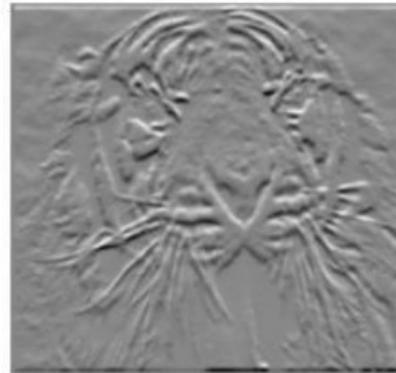
Gradient Magnitude:

$$||\nabla f|| = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2}$$

Gradient Magnitude



$$\frac{\partial f(x, y)}{\partial x}$$



$$\frac{\partial f(x, y)}{\partial y}$$

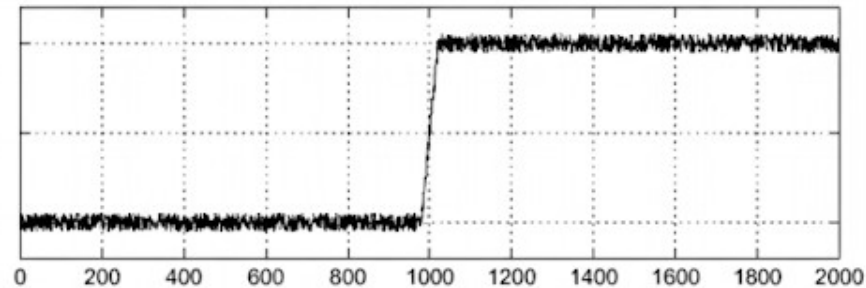


$$||\nabla f|| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Effect of Noise

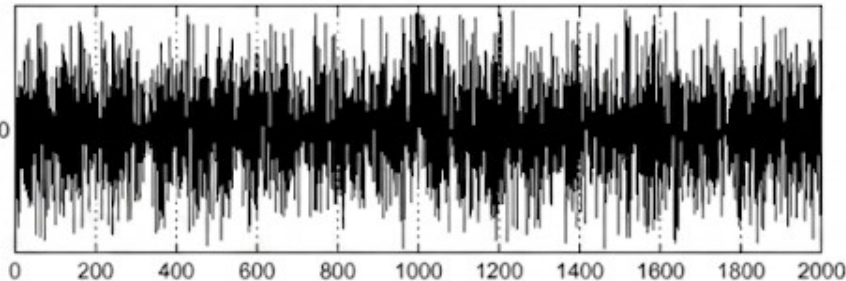
Changes in gray values along a row or column of a noisy image

$f(x)$



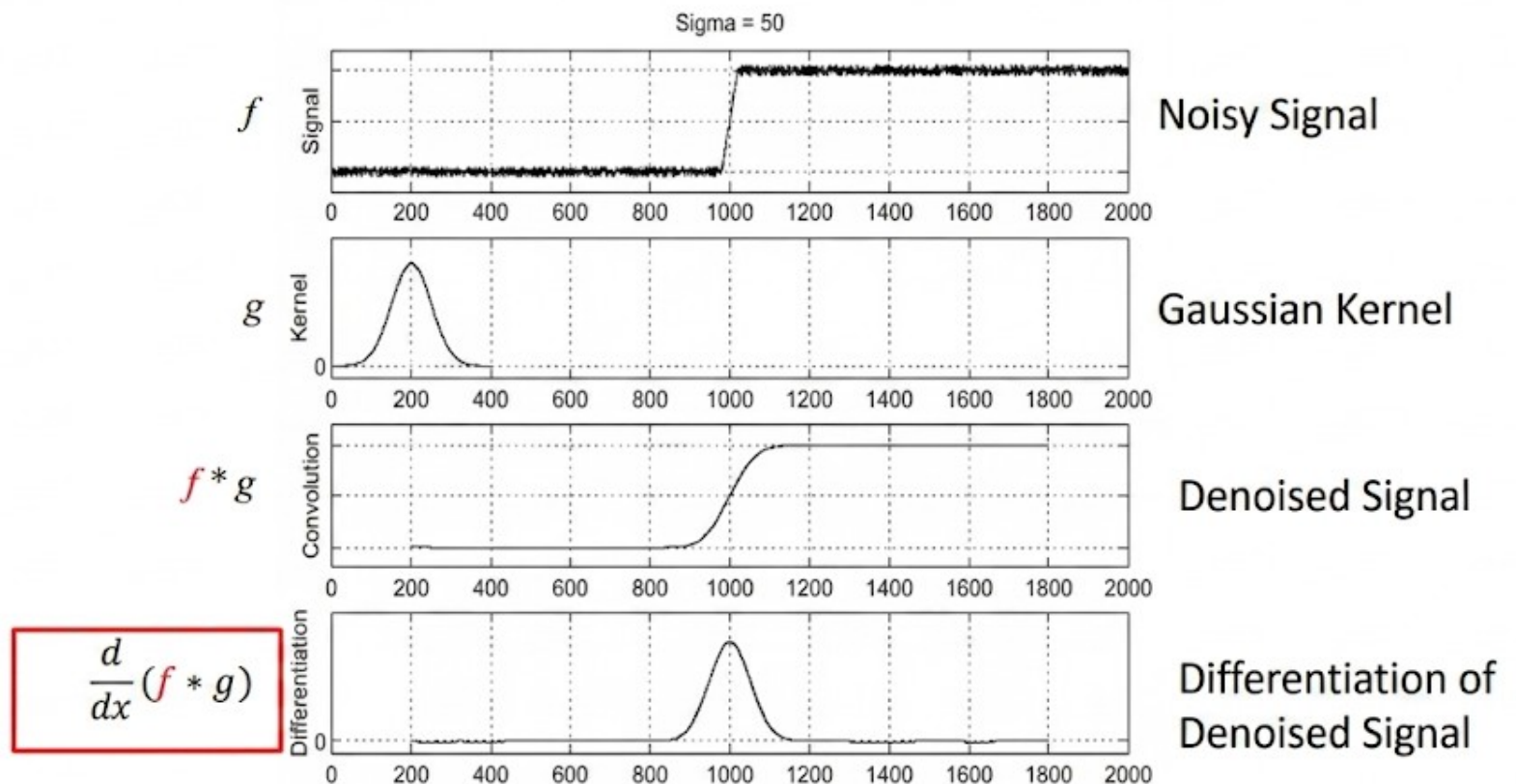
Derivative
result

$\frac{d}{dx}[f(x)]$



Where is the
edge location?

Solution: Smooth First

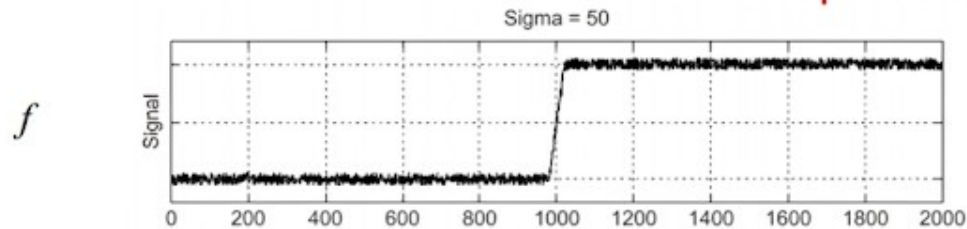


First-Order Partial Derivative of Gaussian

- Differentiation is convolution, and convolution is associative:

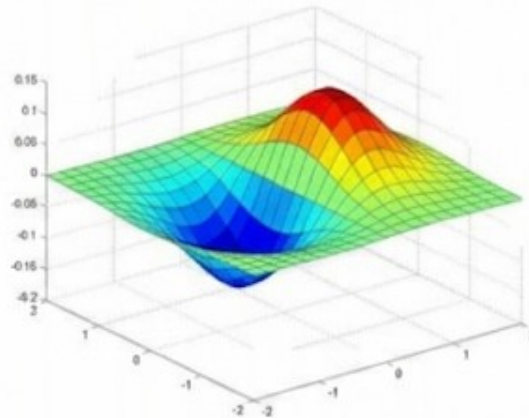
$$\frac{d}{dx}(f * g) = f * \frac{d}{dx}g$$

Saves one image convolution operation!

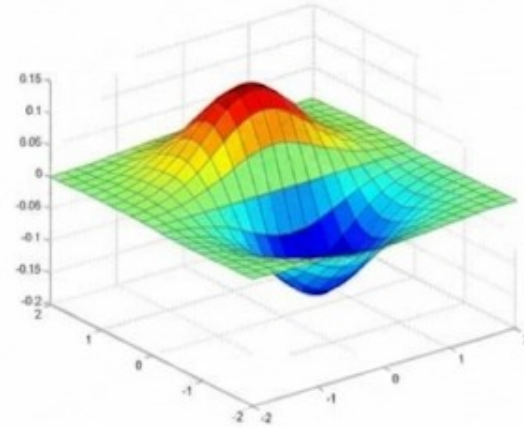


Gaussian First-Order Partial Derivative Convolutional Kernels

Weight Visualization of
Gaussian First-Order
Partial Derivative
Convolutional Kernels

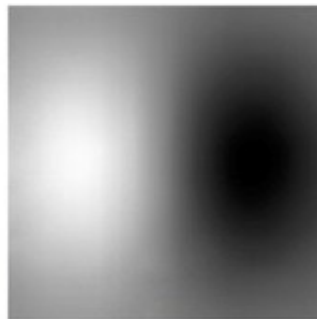


x-direction

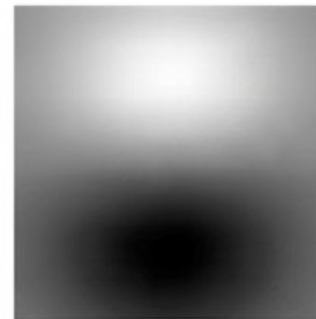


y-direction

Gaussian First-Order
Partial Derivative
Convolutional Kernel

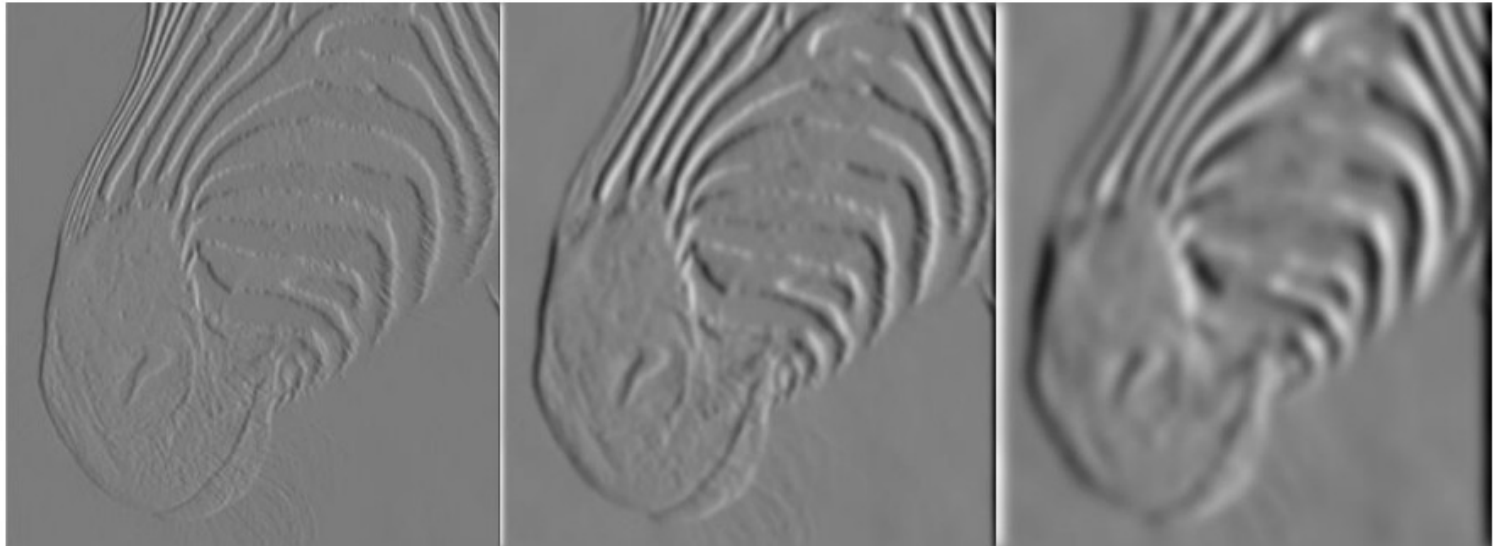


Focus on vertical edges



Focus on horizontal edges

First-order Gaussian Derivative Kernel's Variance Variation



1 pixel

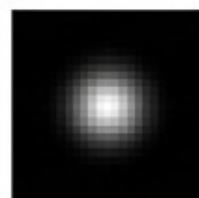
3 pixels

7 pixels

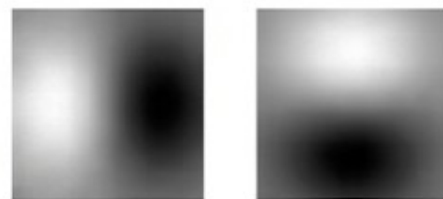
Variance from small to large →

Gaussian Kernel vs. Gaussian First-Order Partial Derivative Kernel

- Gaussian Kernel
 - Eliminates high-frequency components (low-pass filter)
 - Kernel weights cannot be negative
 - Weights sum to 1 (constant regions are not affected by)
- Gaussian First-Order Partial Derivative Kernel
 - Derivative of Gaussian
 - Kernel weights can be negative
 - Weights sum to 0 (no response to constant regions)
 - High response at high-contrast points



Gaussian
Kernel



Gaussian First-Order
Partial Derivative Kernel

Edge Detection Goal

Goal



Original Image

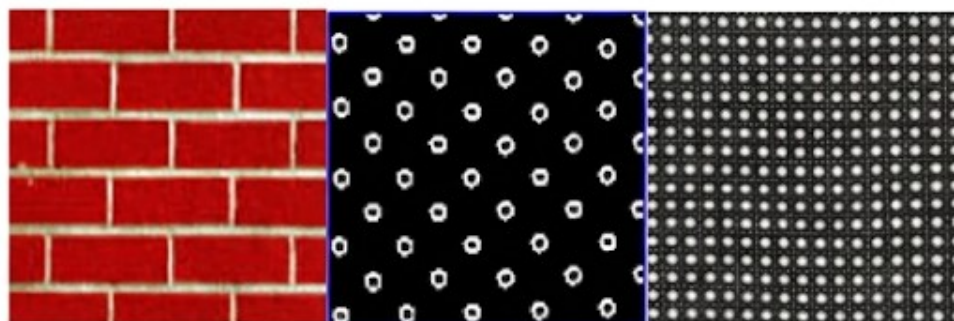


Edge Map

Image Texture Representation

Texture

Regular Texture

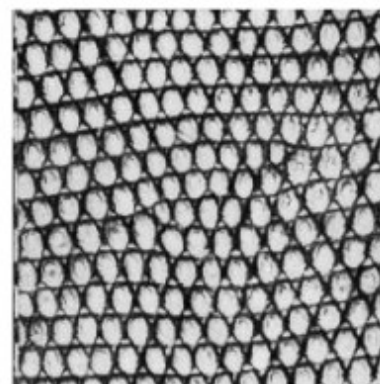
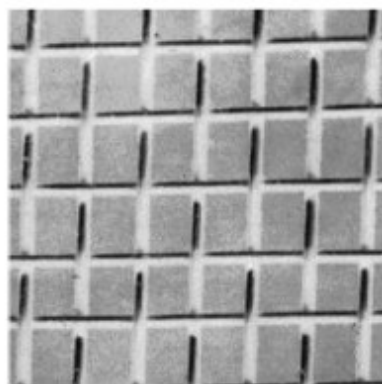
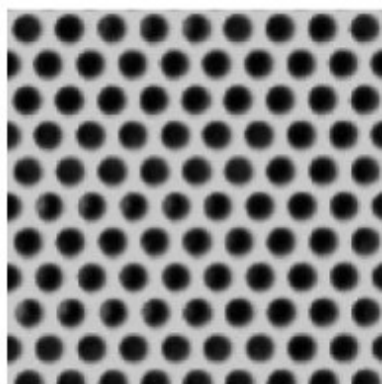


Random Texture



Origin 1: Texture recognition

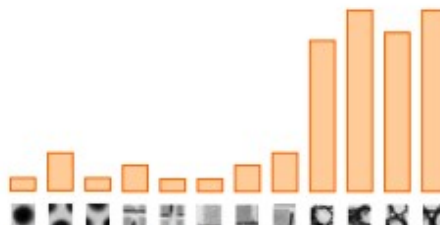
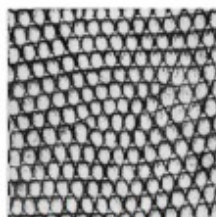
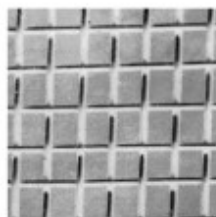
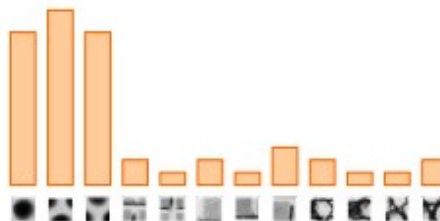
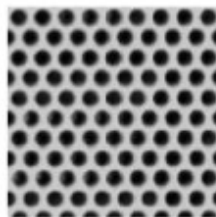
Texture is characterized by the repetition of basic elements or *textons*
For stochastic textures, it is the identity of the textons, not their spatial arrangement, that matters



Julesz, 1981; Cula & Dana, 2001; Leung & Malik 2001; Mori, Belongie & Malik, 2001;
Schmid 2001; Varma & Zisserman, 2002, 2003; Lazebnik, Schmid & Ponce, 2003

Source: Lazebnik

Origin 1: Texture recognition



Julesz, 1981; Cula & Dana, 2001; Leung & Malik 2001; Mori, Belongie & Malik, 2001; Schmid 2001; Varma & Zisserman, 2002, 2003; Lazebnik, Schmid & Ponce, 2003

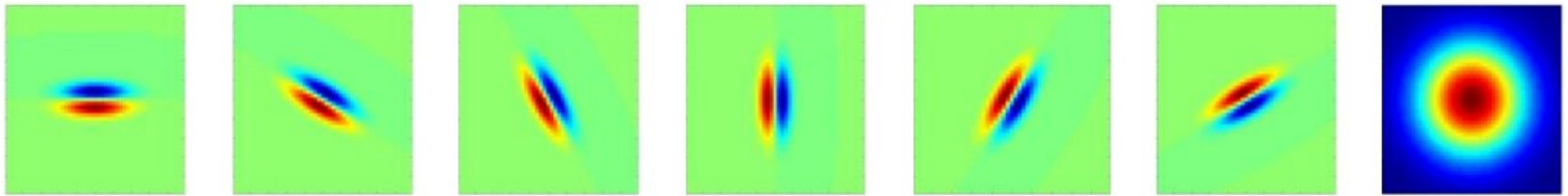
Source: Lazebnik

Convolutional Kernel Group Based Texture Representation Method

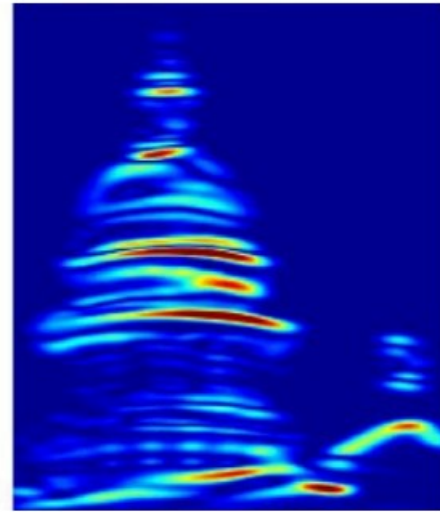
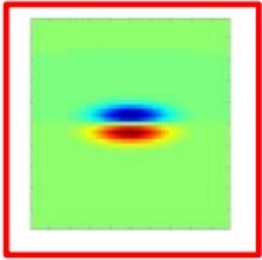
Methodology:

- Utilize convolutional kernel group to extract texture bases from image; utilize the statistical information of bases to represent the texture in the image

Convolution Kernel Bank



Convolution Kernel Bank



Convolution Kernel Bank

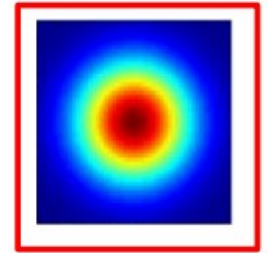
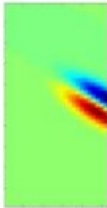
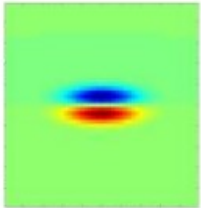
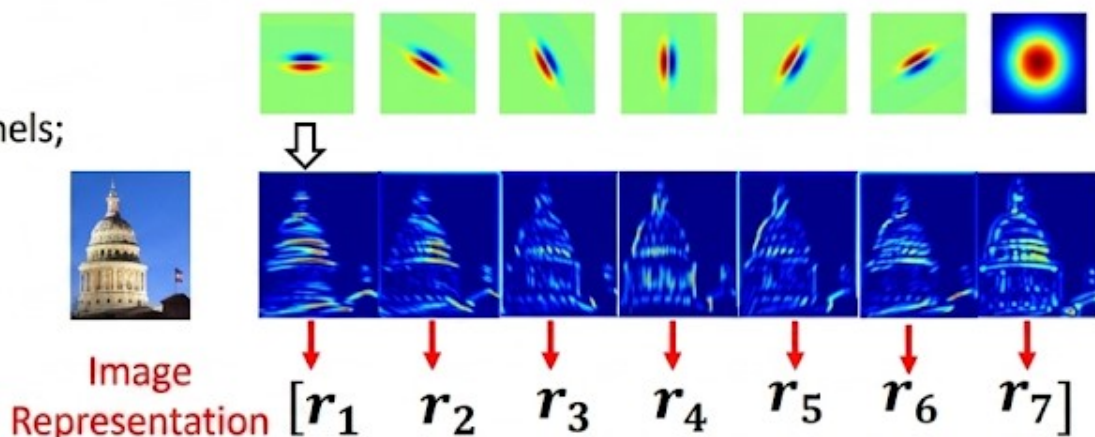


Image Representation Based on a Group of Convolutional Kernels

1. Design a group of convolutional kernels;
2. Use the group of convolutional kernels to perform a convolutional operation and obtain a corresponding set of feature response maps;
3. Use a certain kind of statistical information of the feature response map to represent the image value in the texture.

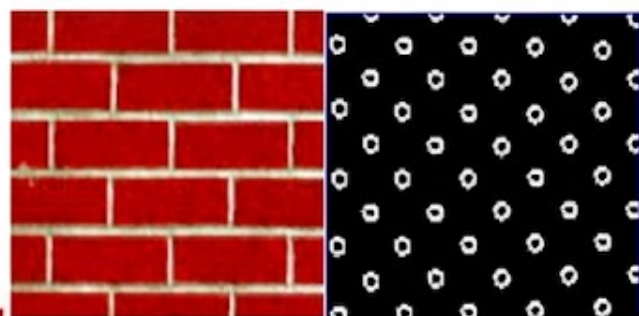


$r_i = [r_{i1}, r_{i2}, \dots, r_{in,n}]$, vector flattened from the i -th feature response map
 r_{ij} reflects whether the i -th convolutional kernel records the texture structure to orae at the j -th position of the image

Texture Classification Task

- Ignore base element positions
- Focus on which base elements appeared

Textures and the frequency of base elements' appearances



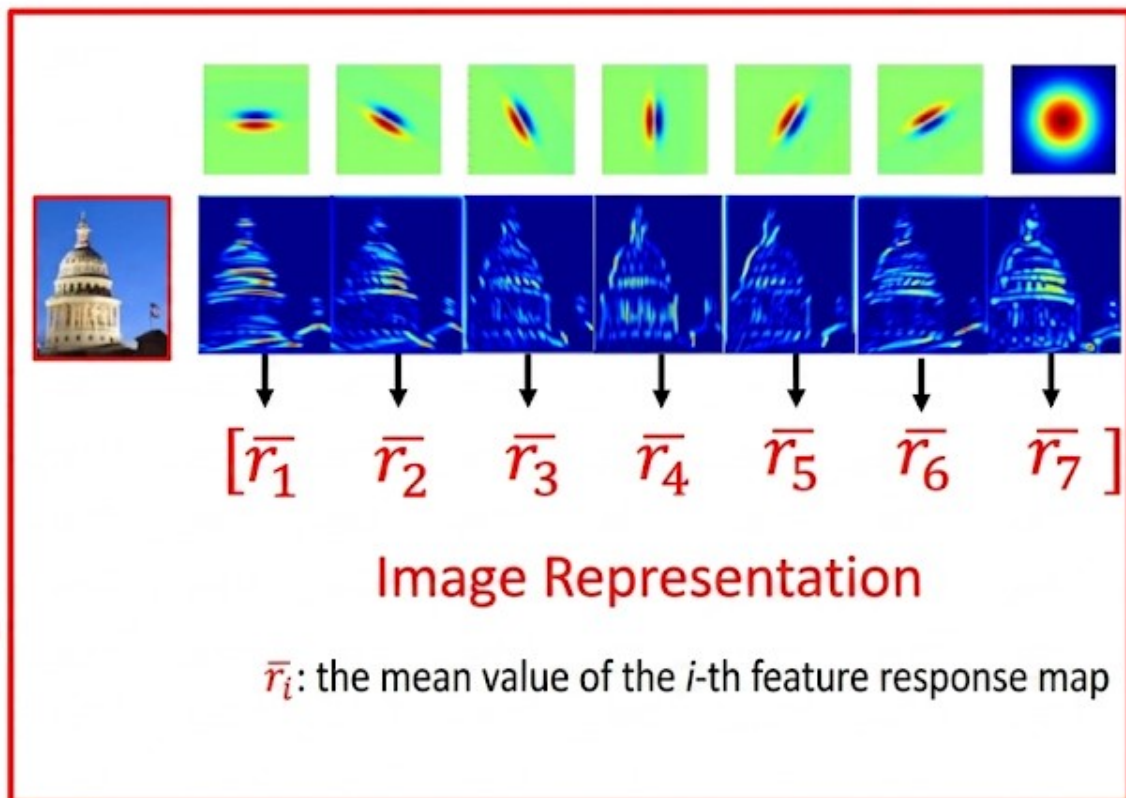
Regular Textures



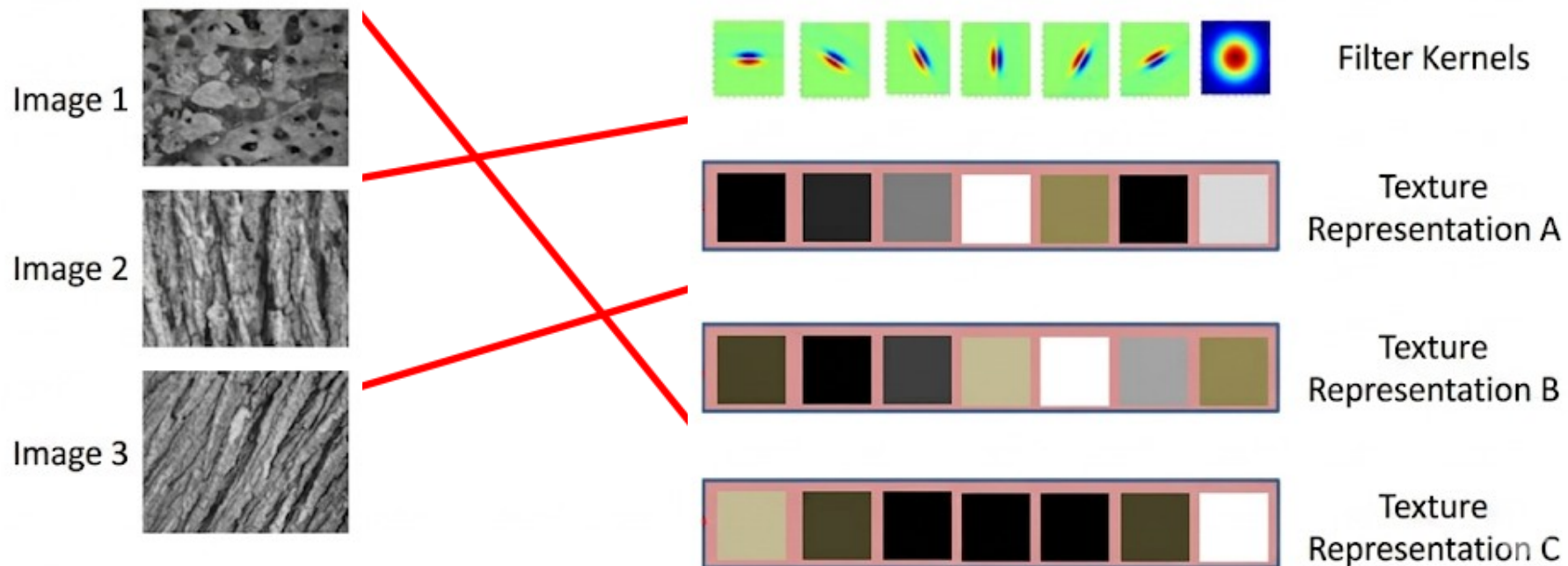
Random Textures

Image Representation Based on a Group of Convolutional Kernels

1. Design a group of convolutional kernels;
2. Apply the group of convolutional kernels to an image, performing convolutional operations to obtain a set of corresponding feature response maps;
3. Utilize a certain statistical information from the feature response maps to represent the texture in the image.



Texture Representation Game

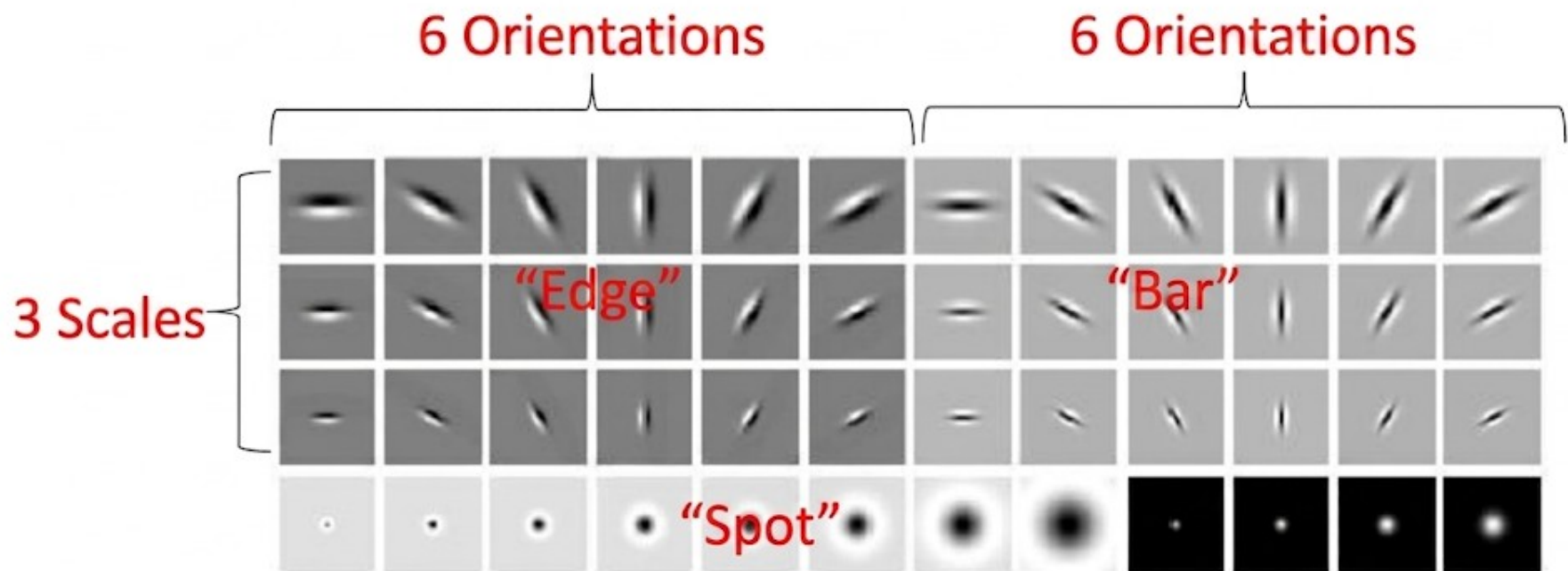


Convolutional Kernel Group Design

Design Focus:

- Kernel Type (edge, bar, and spot)
- Kernel Scale (3-6 scales)
- Kernel Orientation (6 angles)

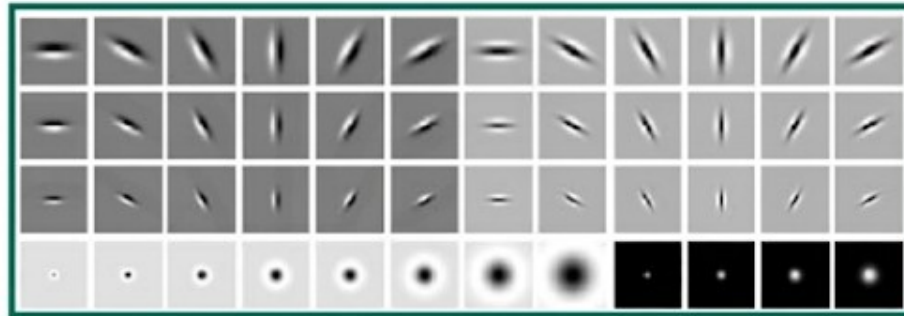
A Practical Example



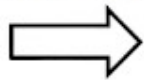
Convolutional Kernel Group

A Practical Example

Filter Bank



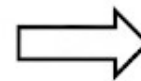
Apply
Filter Bank



...



48 Feature Response Maps



$[\bar{r}_1 \quad \dots \quad \bar{r}_{48}]$

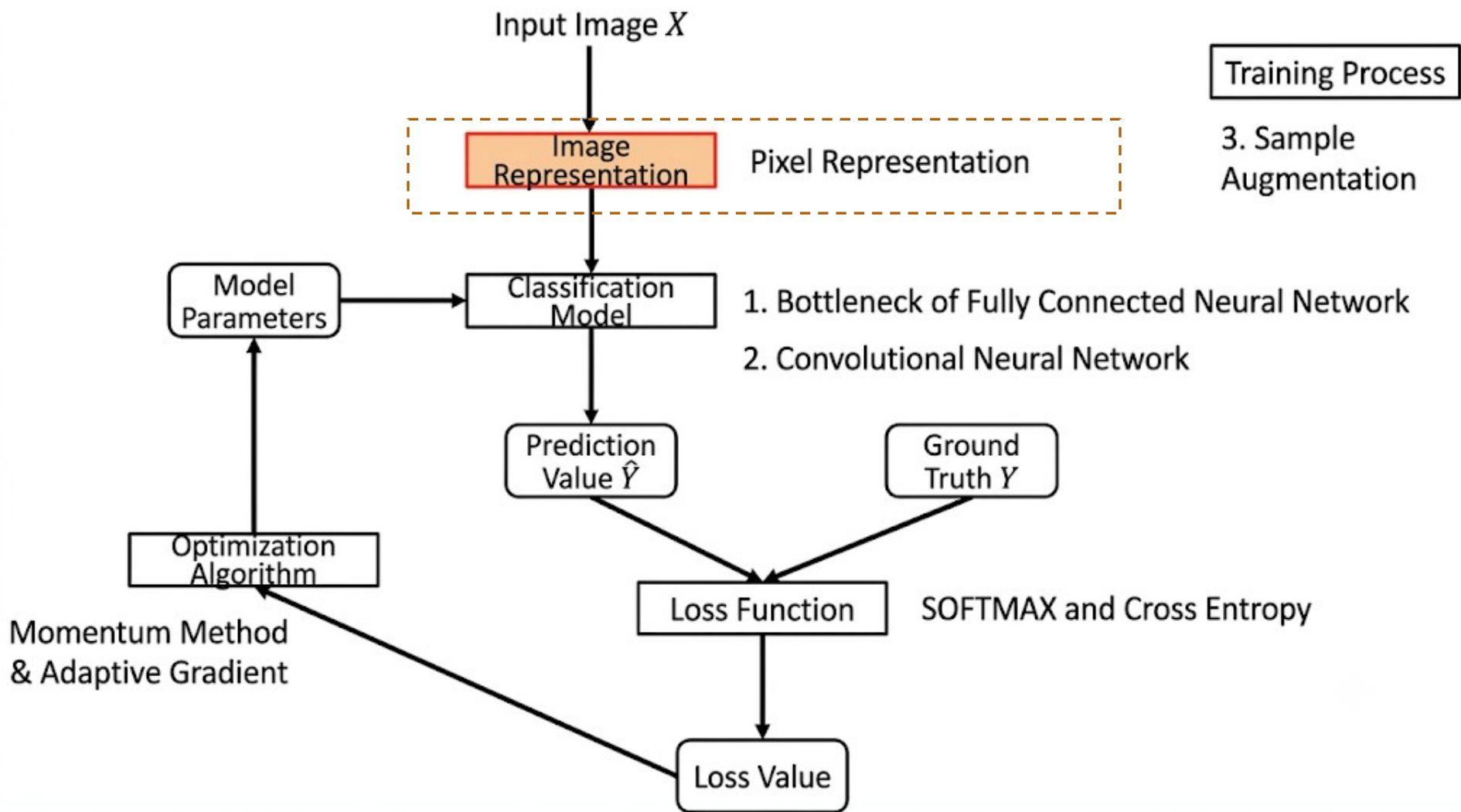
48-dimensional
Vector Representation

$\bar{r}_i$: the average value
of the i-th feature
response map

Summary

1. Design a set of convolutional kernels;
2. Using the set of convolutional kernels taps on images
Perform convolutional operations to obtain corresponding sets of feature response maps;
3. Utilize certain statistical information of the feature response maps to represent the textures in the images.

Convolutional Neural Networks

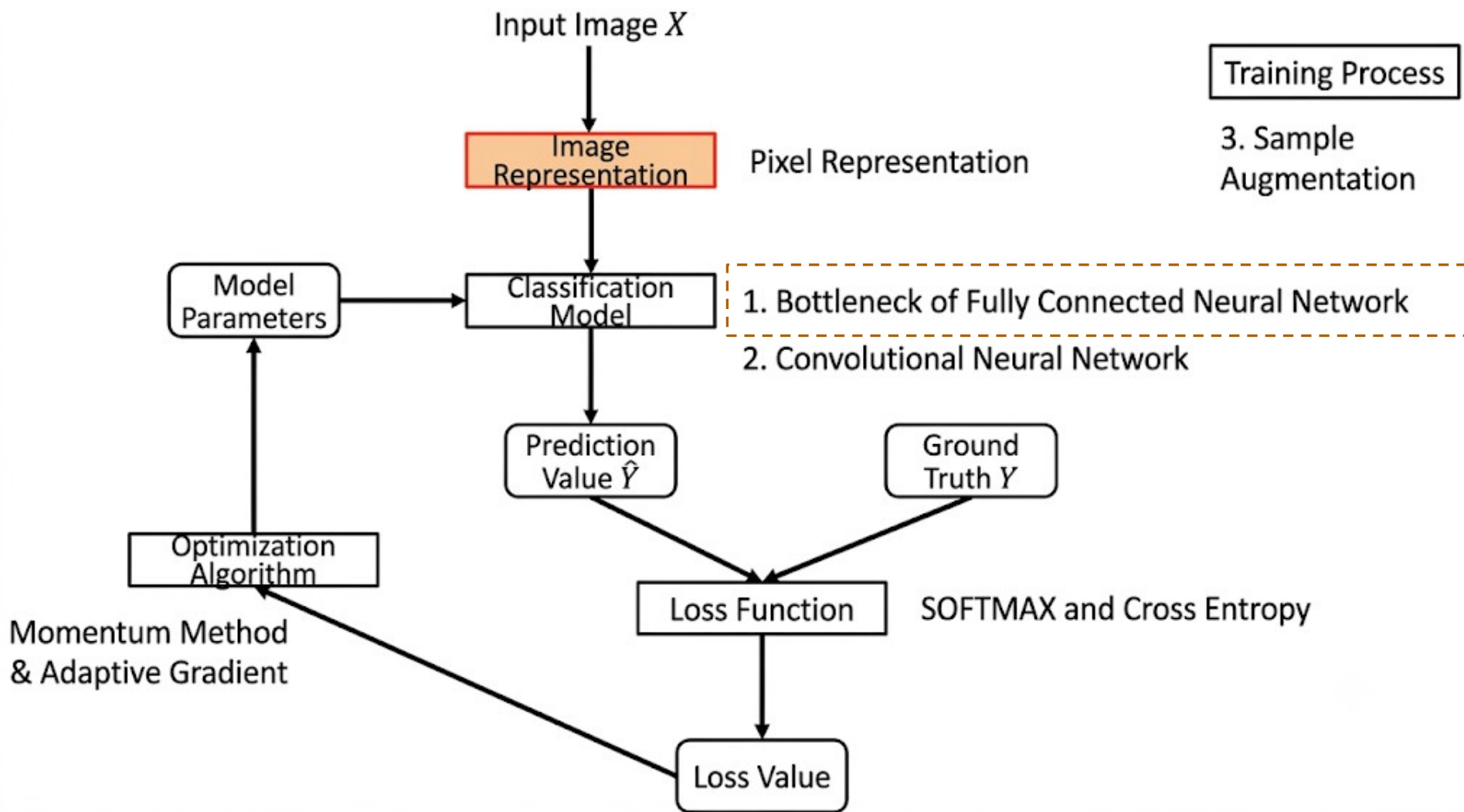


How to Represent an Image?

Directly use **raw pixel values as features**, flatten into a column vector.

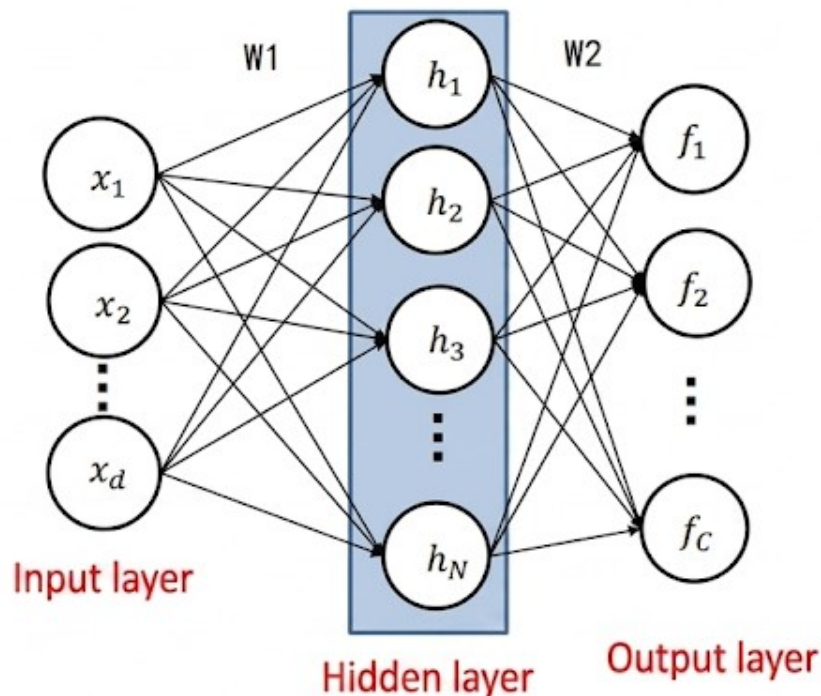
$$\begin{matrix} \begin{bmatrix} 76 & 32 \\ 4 & 157 \end{bmatrix} \\ \text{Image} \end{matrix} \xrightarrow{\text{Flatten the matrix into a column vector}} x = \begin{bmatrix} 76 \\ 32 \\ 4 \\ 157 \end{bmatrix}$$

Each image on cifar10 is represented as a 3072 ($32 \times 32 \times 3$) dim. vector



The Bottleneck of Fully Connected Neural Networks

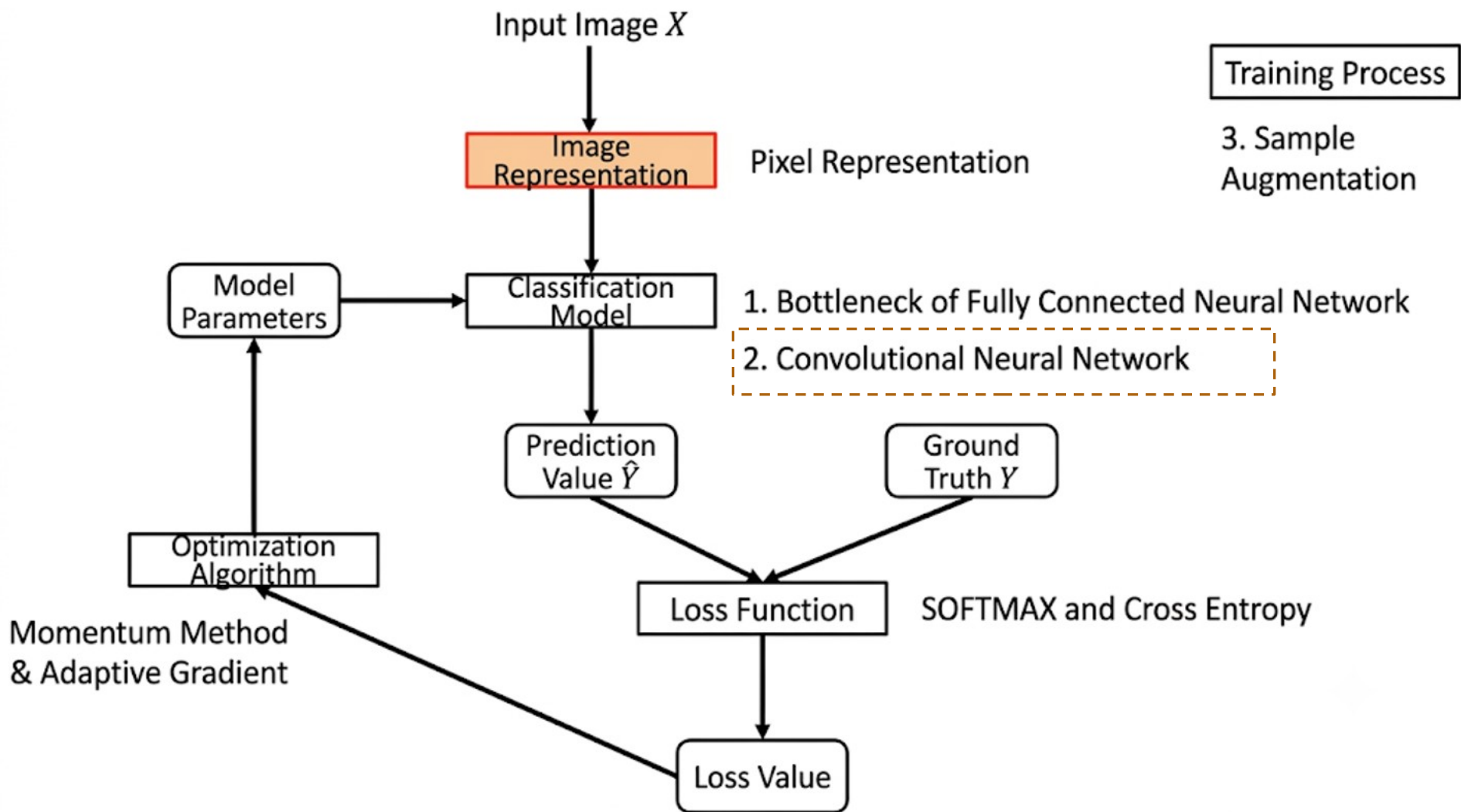
Two-layer fully connected network



Question: CIFAR10 image size $32 \times 32 \times 3$,
how many weights per hidden layer neuron?
Answer: $32 \times 32 \times 3 = 3072$

Question: If image size $200 \times 200 \times 3$, how
many weights per hidden layer neuron?
Answer: $200 \times 200 \times 3 = 120,000$

Only suitable for processing small images!



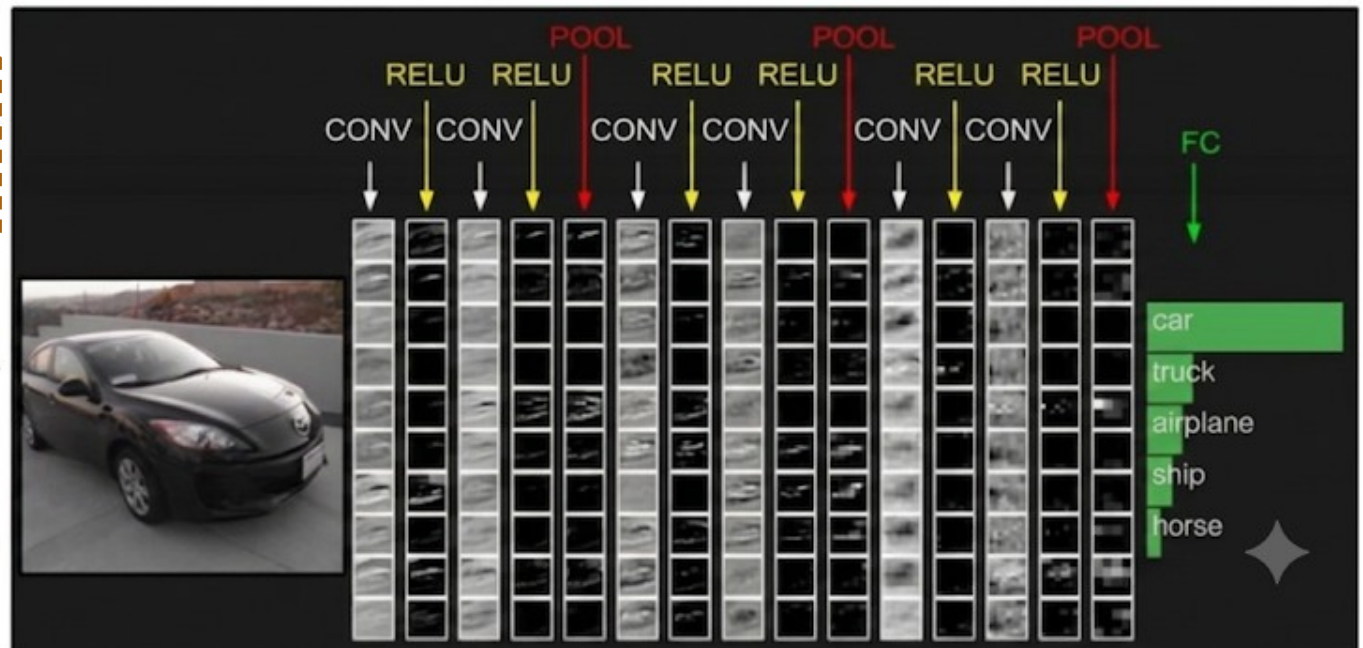
Convolutional Neural Network

CONV — Convolutional Layer

RELU — Activation Layer

POOL — Pooling Layer

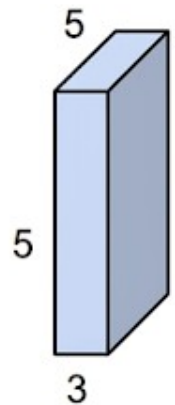
FC — Fully Connected Layer



Convolutional Kernels in Convolutional Networks

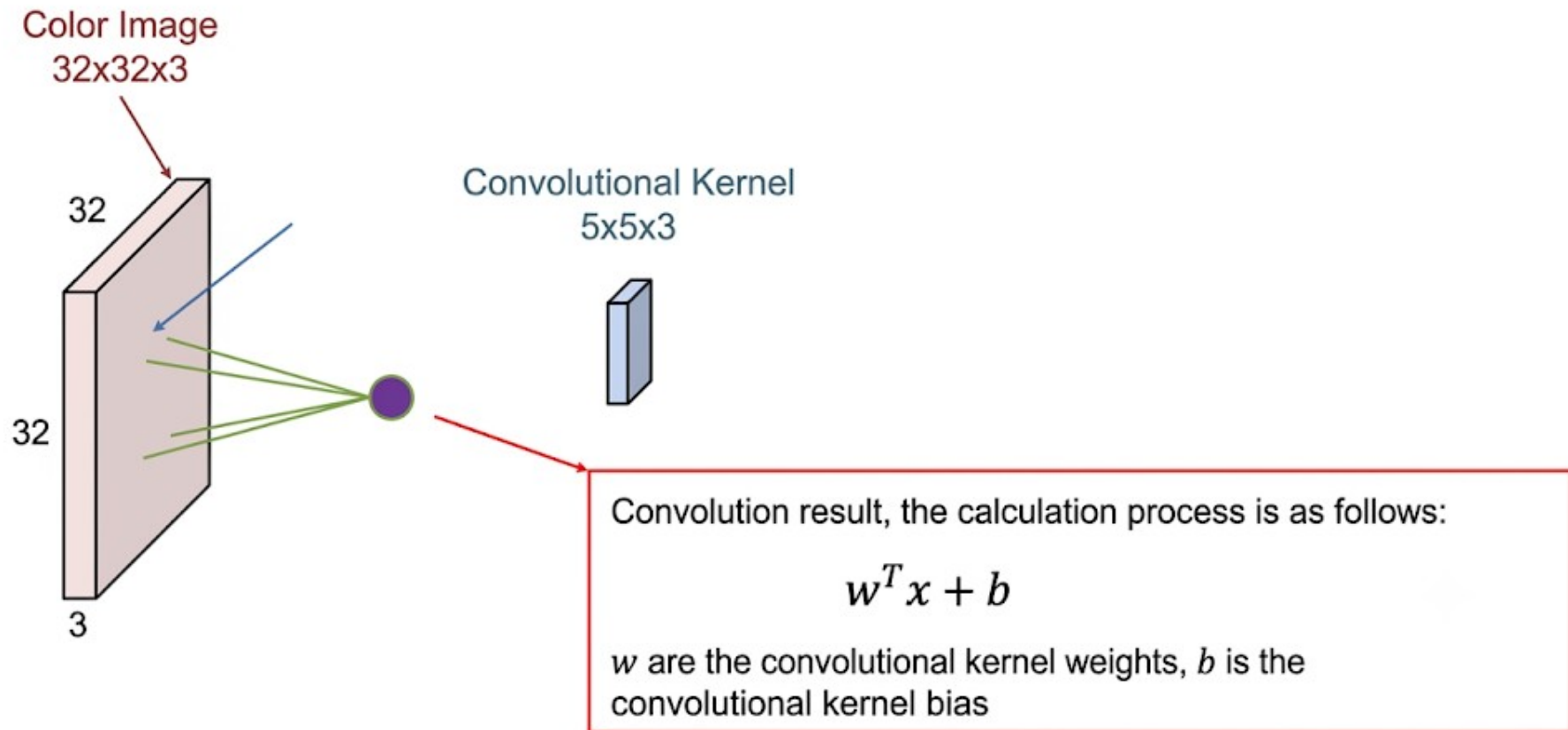
Convolutional Kernel:

- Not only has width and height, but also depth, often written in the following form:
Width x Height x Depth
- Convolutional kernel parameters not only include the weights stored in the kernel, but also a bias value

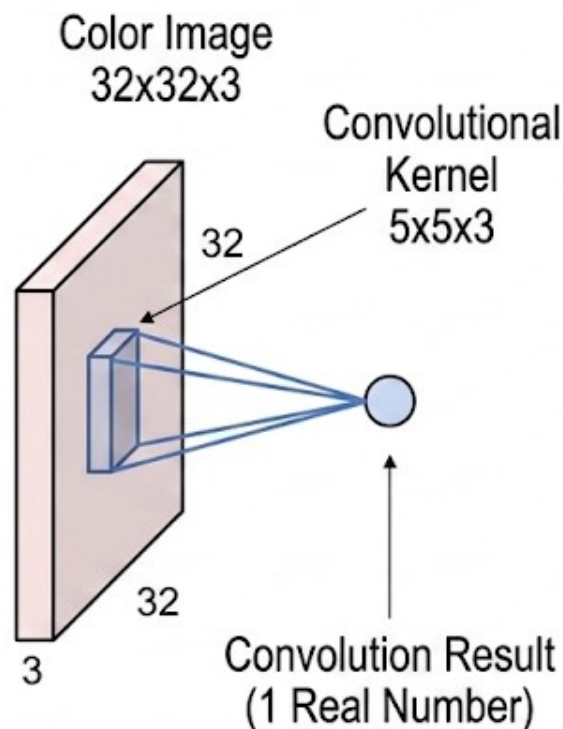


5x5x3 Convolutional
Kernel

Convolution Operation in Convolutional Networks



Convolution Operation in Convolutional Networks



Calculation Process:

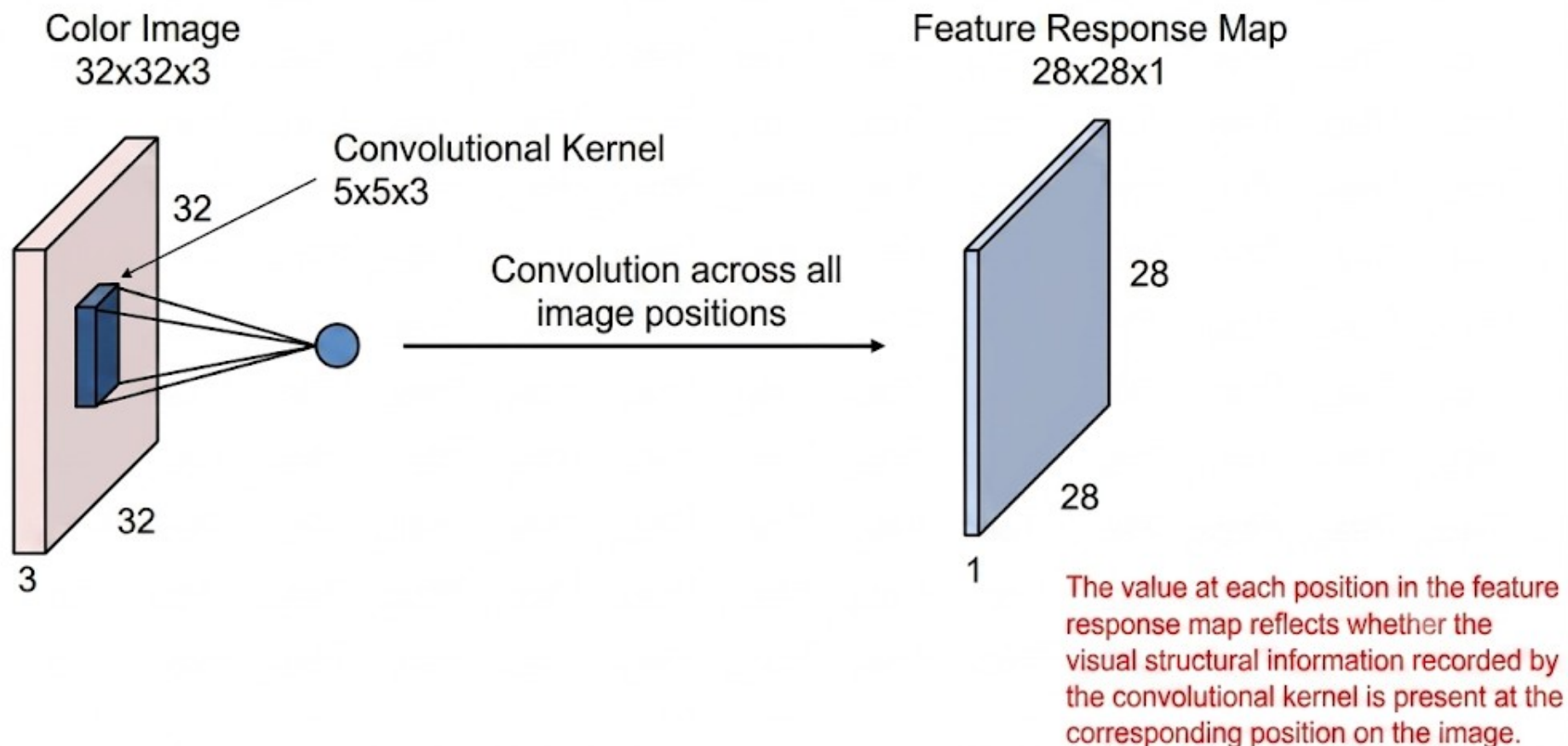
- Flatten the convolutional kernel into a 5x5x3 vector; simultaneously, flatten the covered image region into a 5x5x3 vector in the same manner.
- Calculate the dot product of the two.
- ✓ Add the bias value to the dot product result.

Mathematical Formula:

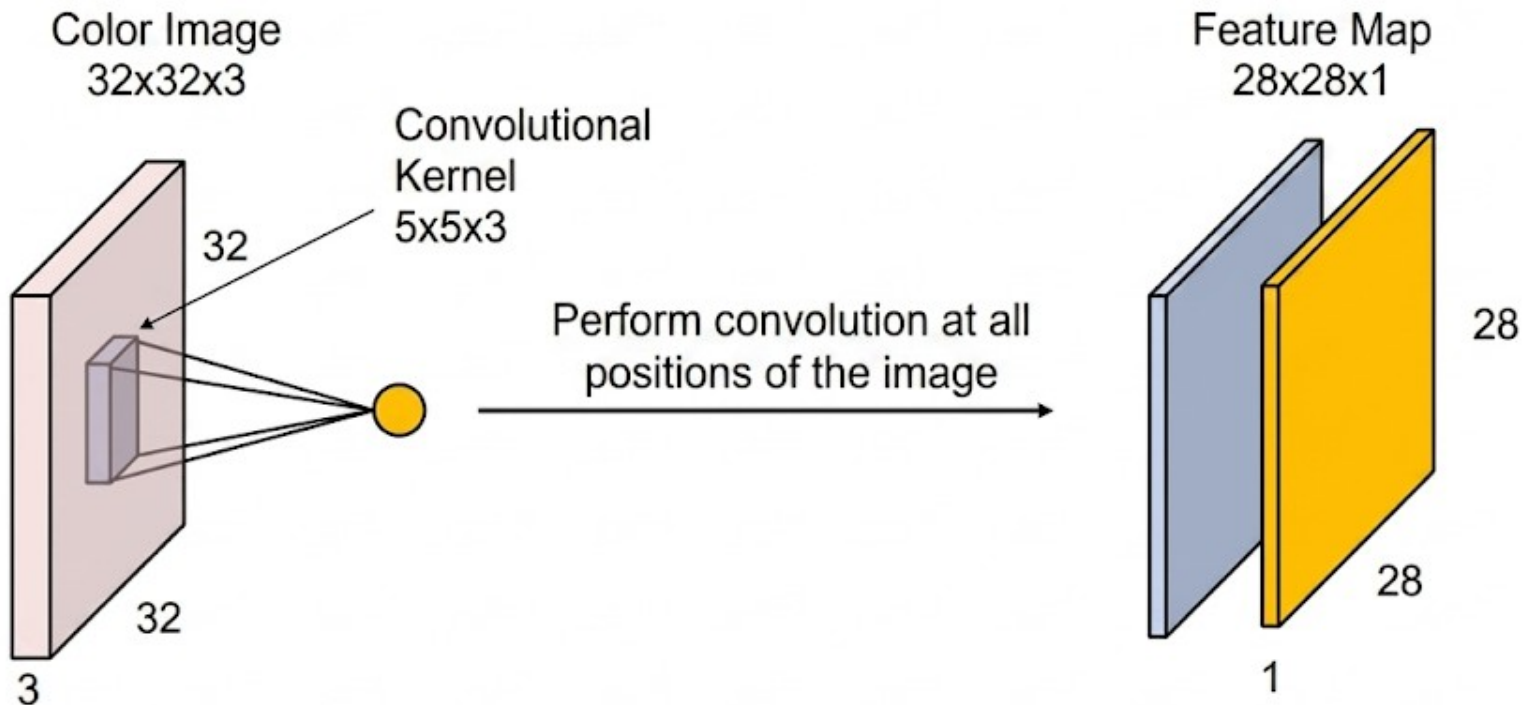
$$w^T x + b$$

w are the convolutional kernel weights, b is the convolutional kernel bias

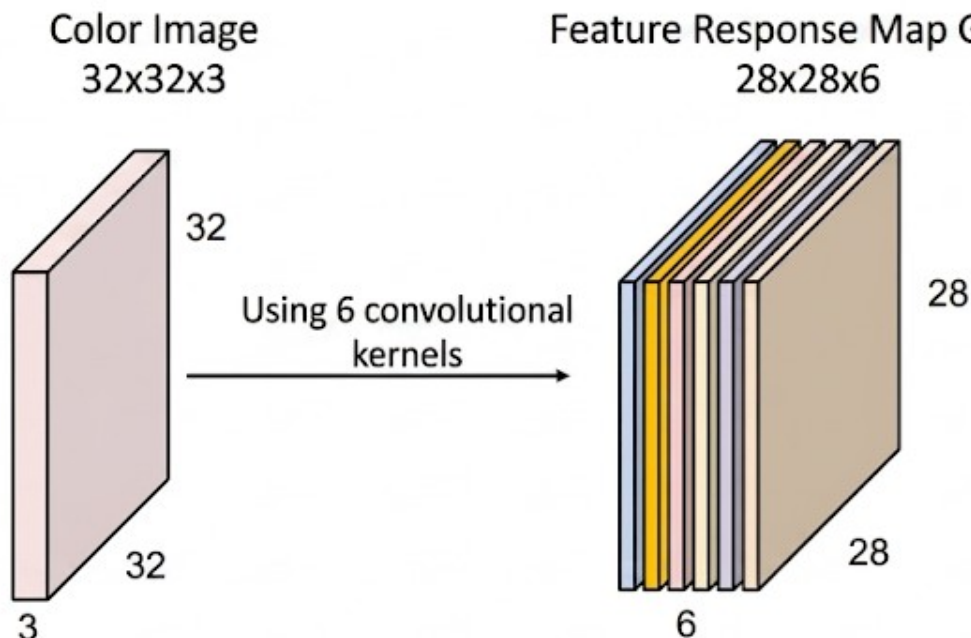
Convolution Operation in Convolutional Networks



Convolutional Layer



Convolutional Layer



- The depth of the feature response map group equals the number of convolutional kernels;
- Different feature response maps reflect the input image's response results to different convolutional kernels;
- Values at different positions on the same feature response map represent the response results of different positions on the input image to the same convolutional kernel.

Note:

The convolutional layer input is not limited to images; it can be any three-dimensional data matrix; The depth of the convolutional kernel for this layer is required to be **consistent** with the depth of the input three-dimensional matrix.

Convolution Stride

In a convolutional neural network, the convolutional kernel can perform convolution operations at designated intervals. This interval is the **convolution stride**.

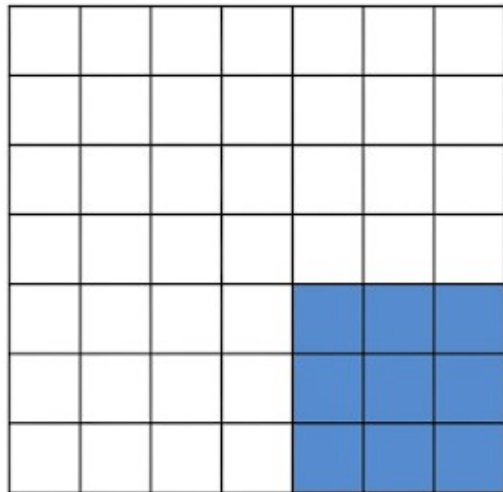


Image Size: 7x7

Kernel Size: 3x3

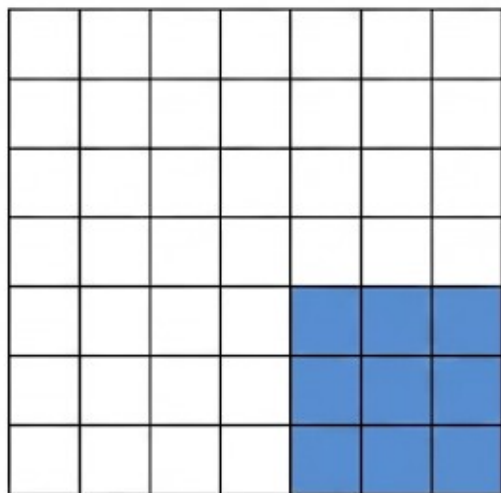
Convolution Stride: 1

Convolution Result: **5x5**

Convolution Stride

In convolutional neural networks, the convolutional kernel (filter) can perform convolutional operations at specified intervals.

This interval is called the **convolution stride**.



Convolution Output: 3x3

Image Size: 7x7

Convolution Kernel Size: 3x3

Convolution Stride: 2

Input Data Matrix Size: $W1 \times H1$

Output Feature Map Group Size: $W2 \times H2$

$W2$ and $W1$ Relationship is as follows:

$$W2 = (W1 - F) / S + 1$$

$$H2 = (H1 - F) / S + 1$$

F — Convolution Kernel Size

S — Convolution Stride

Padding

In convolutional neural networks, the most commonly used padding method is **zero padding**.

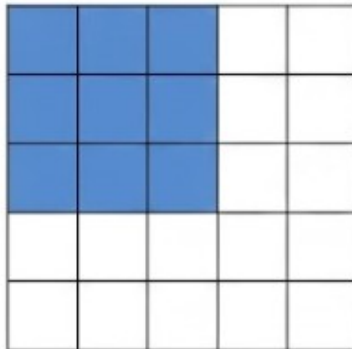


Image Size: 5x5

Kernel Size: 3x3

Stride: 1

Zero Padding: 0

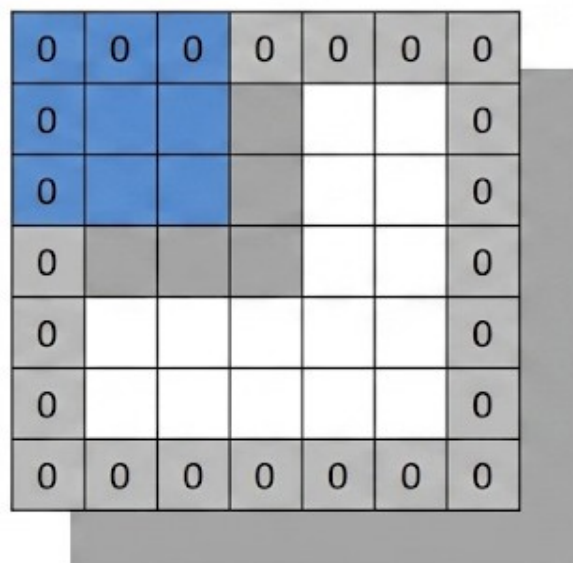
Output Size: 3x3

Empty Pixel



Edge Padding

In convolutional neural networks, the most common padding method is **zero padding**.



0	0	0	0	0	0	0
0						0
0						0
0						0
0						0
0						0
0	0	0	0	0	0	0

Image Size: 5x5

Kernel Size: 3x3

Stride: 1

Zero Padding: 1

Output Size: 5x5

特征响应图组尺寸计算

给定输入数据矩阵时，影响输出的特征图组尺寸大小的因素：

- 卷积核的宽、高；
- 是否采用边界填充操作；
- 卷积步长；
- 该层的卷积核个数。

输入数据矩阵尺寸： $W1 \times H1 \times D1$

输出特征图组尺寸： $W2 \times H2 \times D2$

特征图组尺寸计算如下：

$$W2 = (W1 - F + 2P) / S + 1$$

$$H2 = (H1 - F + 2P) / S + 1$$

$$D2 = K$$

F——卷积核尺寸

S——卷积步长

P——零填充数量

K——卷积核个数

卷积层的4个超参数

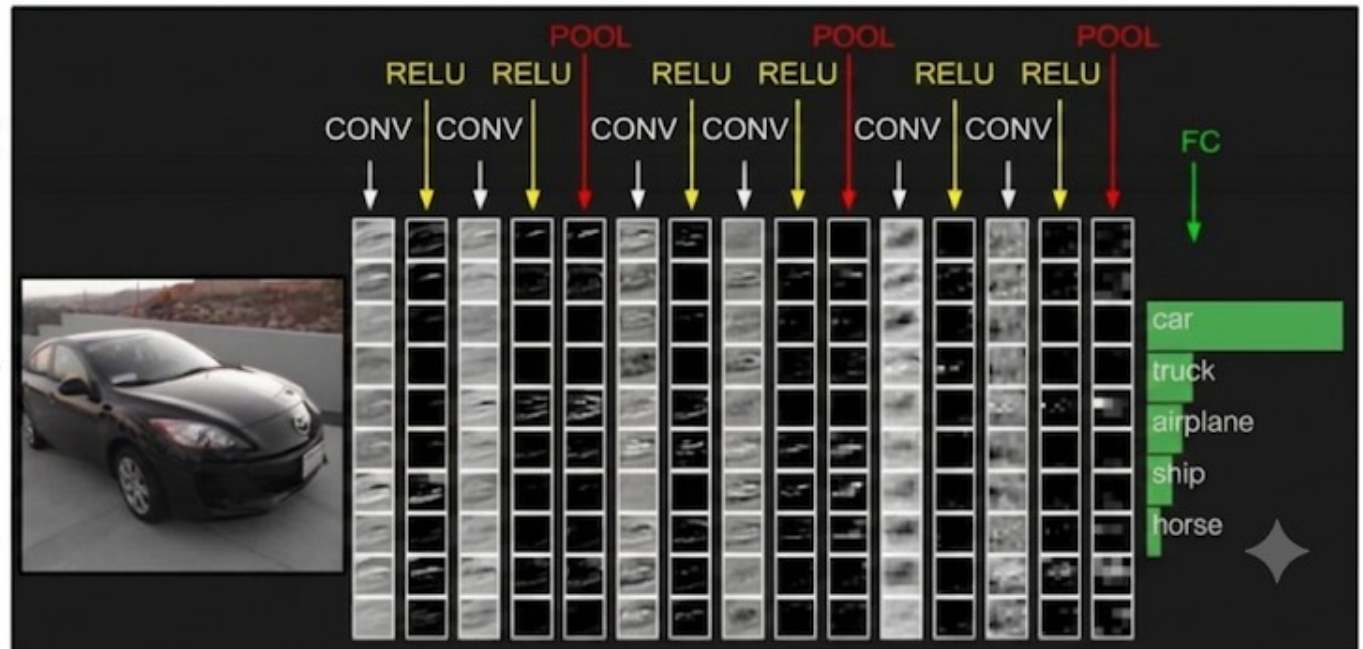
Convolutional Neural Network

CONV — Convolutional Layer

RELU — Activation Layer

POOL — Pooling Layer

FC — Fully Connected Layer



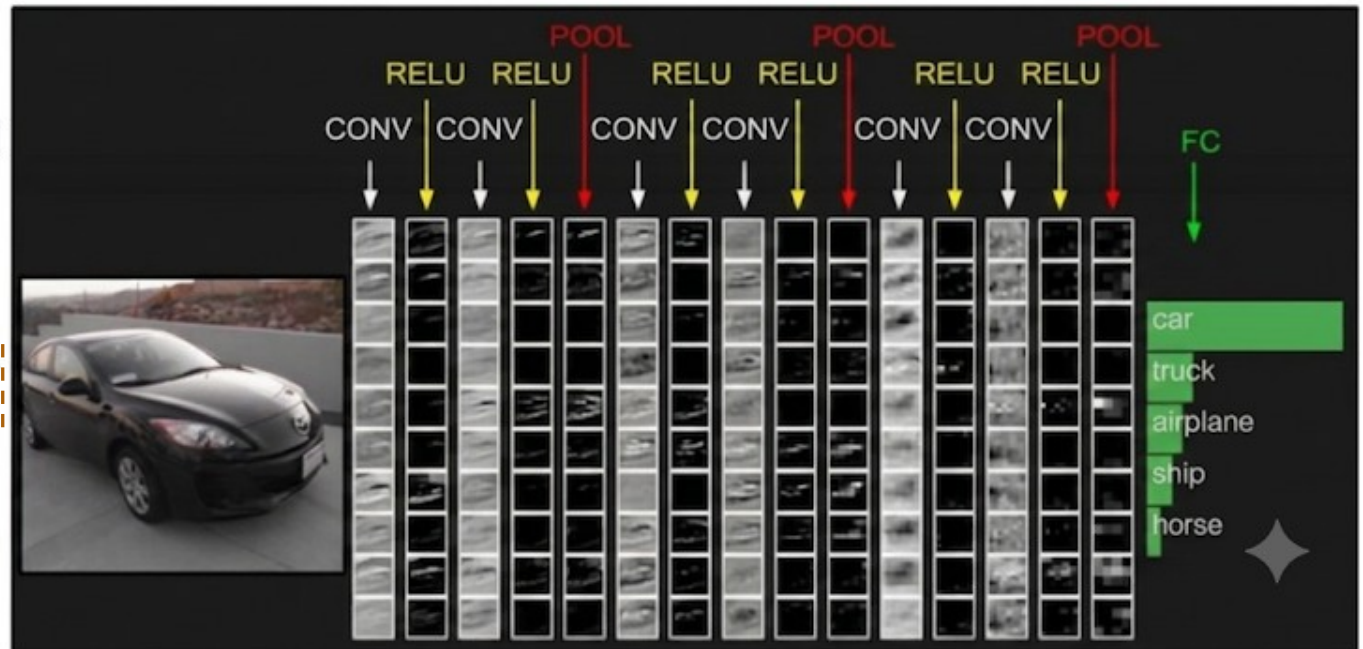
Convolutional Neural Network

CONV — Convolutional Layer

RELU — Activation Layer

POOL — Pooling Layer

FC — Fully Connected Layer

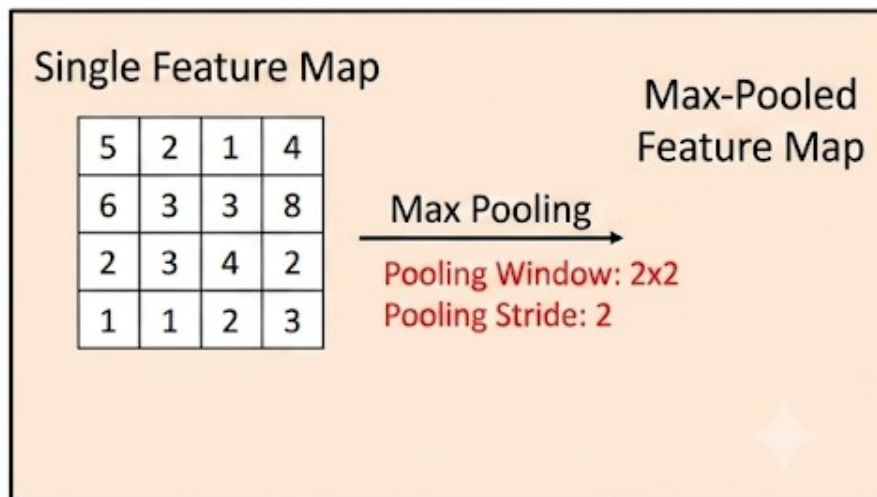
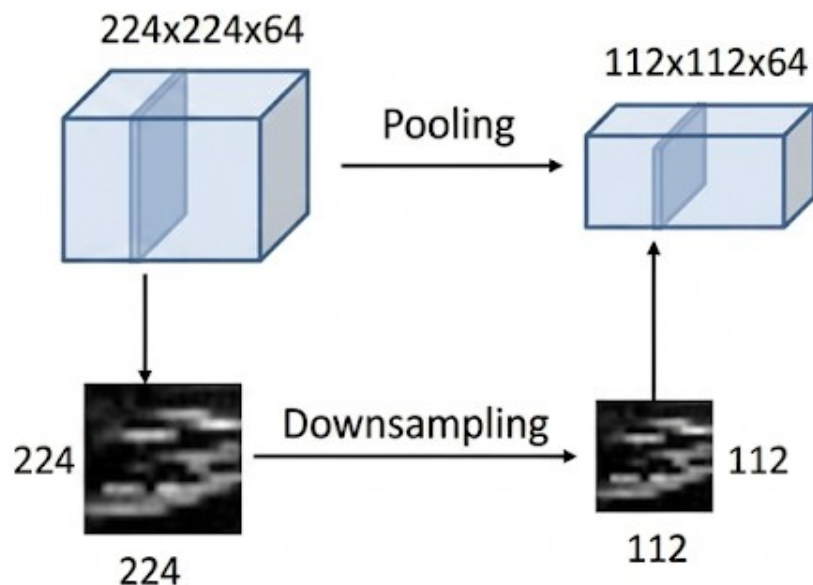


Pooling Operation

- **Role of Pooling:** It is performed independently on each feature response map (or activation map), to reduce the width and height of each feature response map in the set, decrease the number of parameters in subsequent convolutional layers, reduce computational resource consumption, and thus control overfitting.
- **Pooling Operation:** Pooling on a specific region of a feature response map means selecting a single value to represent that entire region.
- **Common Pooling Operations:**
 - ❑ **Max Pooling**—Uses the maximum value within the region to represent it;
 - ❑ **Average Pooling**—Uses the average (mean) value of all values within the region as the representative.
- **Hyperparameters of Pooling Layers: Pooling Window and Pooling Stride**

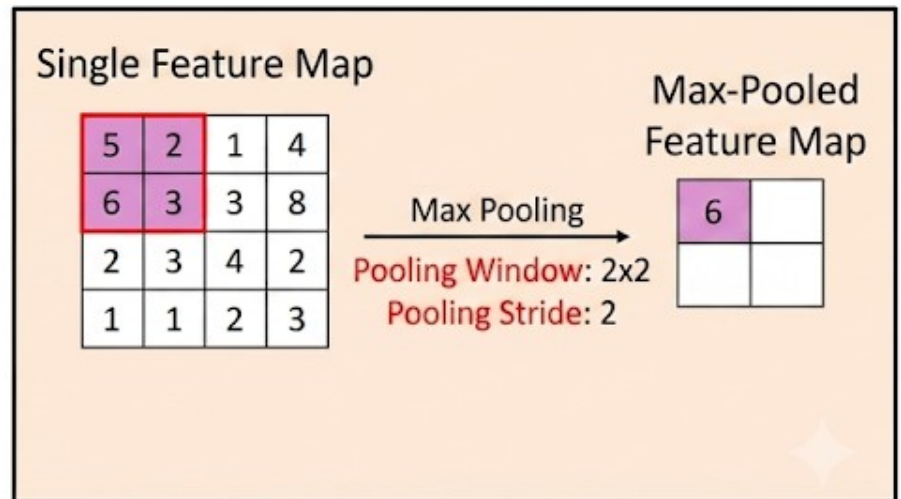
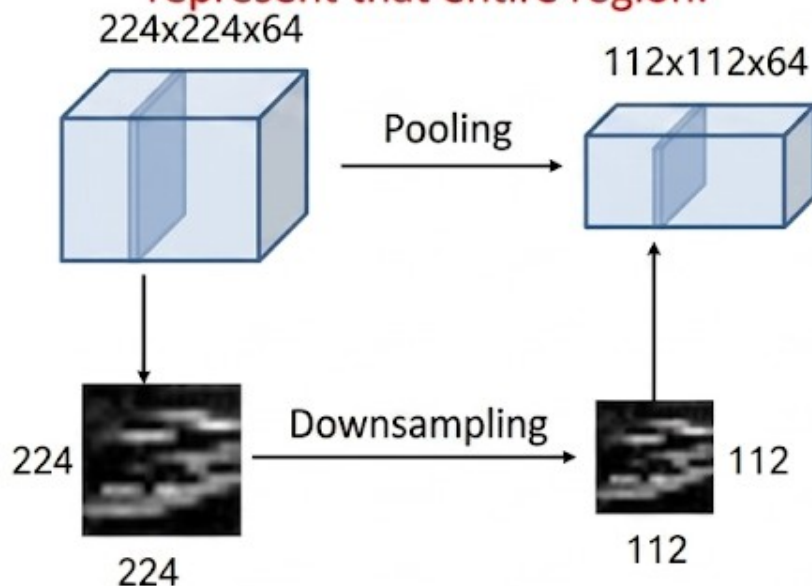
Pooling Operation Example

- The pooling operation is performed independently on each feature response map;
- Pooling over a area of a feature response map means selecting a value in that area to represent the entire area.



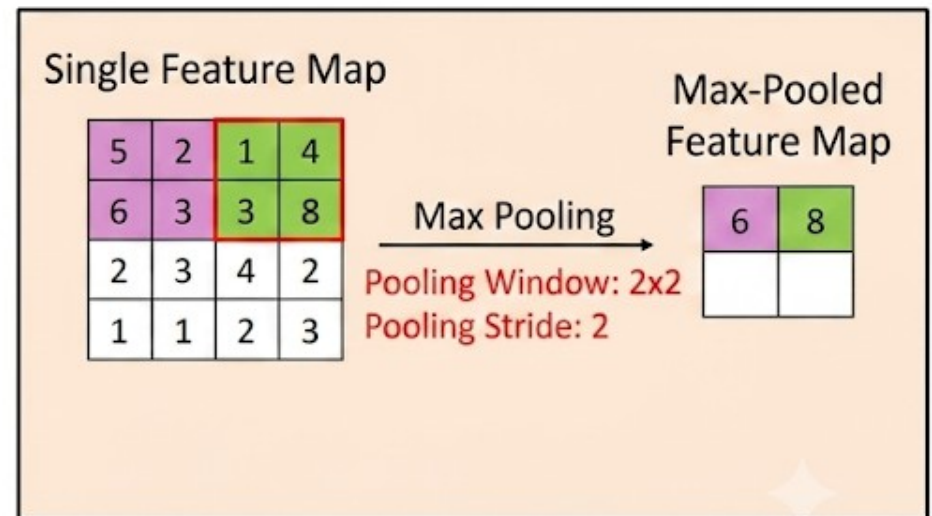
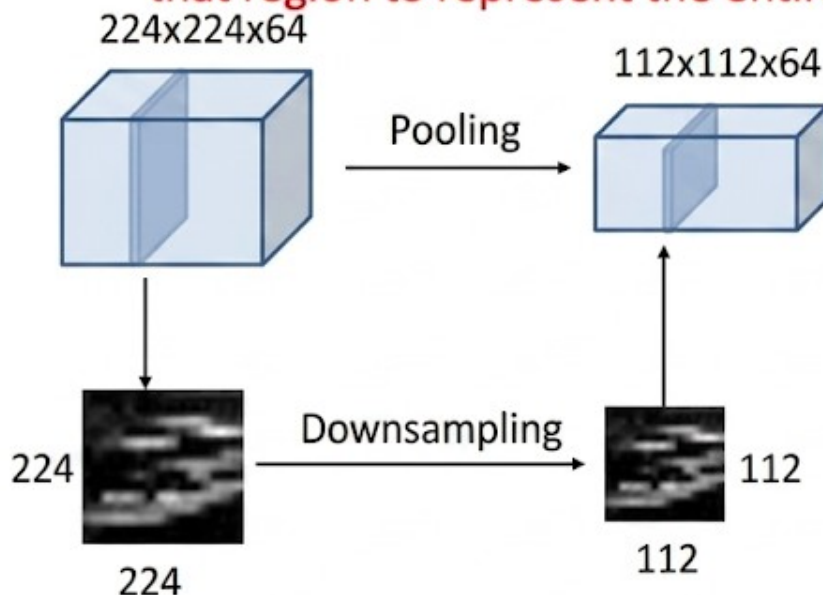
Pooling Operation Example

- Pooling operations are performed independently on each feature response map;
- Pooling on a region of a feature response map means specifying a value to represent that entire region.



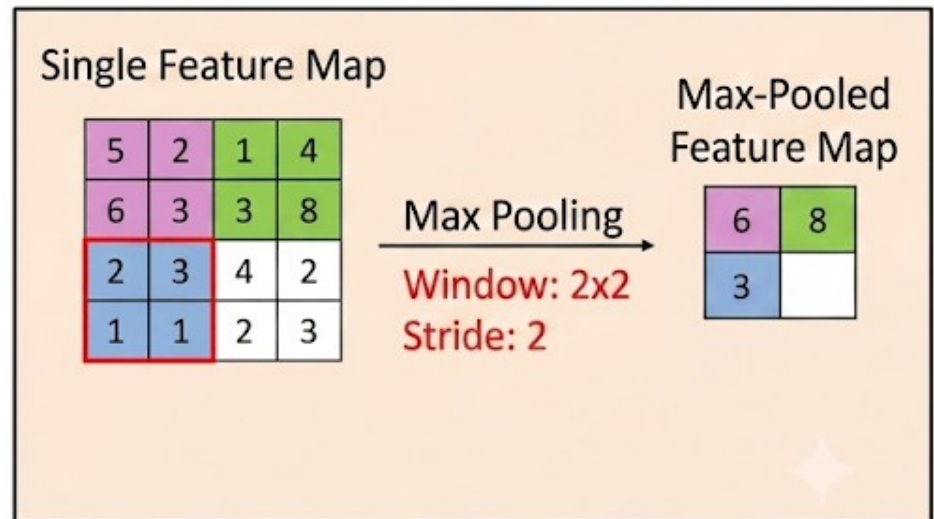
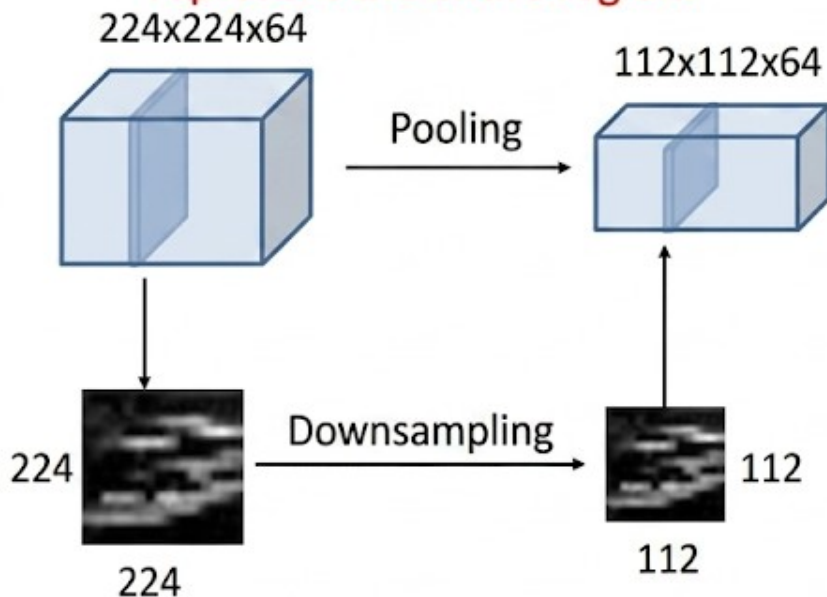
Pooling Operation Example

- The pooling operation is performed independently on each feature response map;
- Pooling on a specific region of a feature response map means selecting a value in that region to represent the entire region.



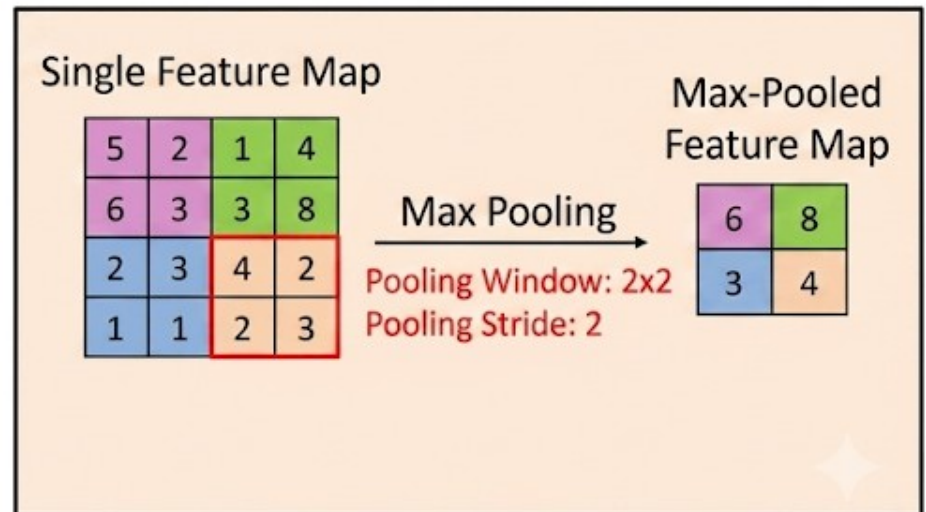
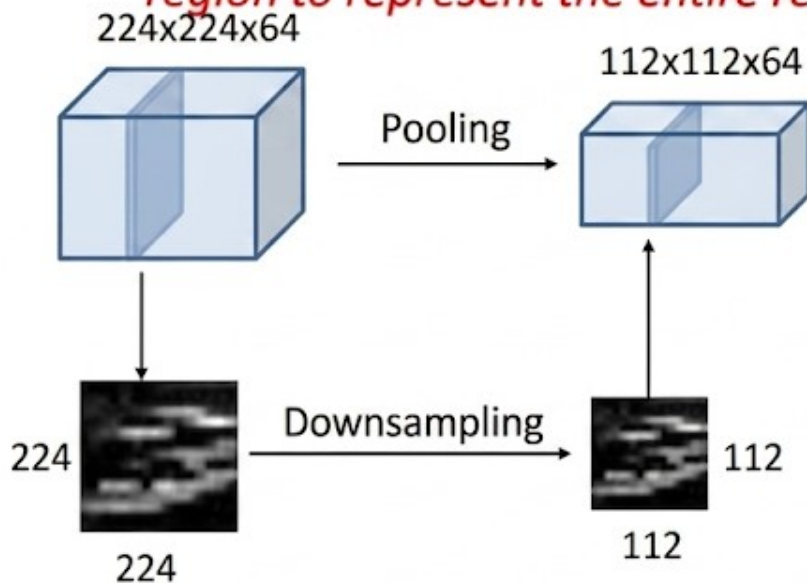
Pooling Operation Example

- Pooling operations are performed independently on each feature response map;
- Pooling on a region of a feature response map means specifying a single value to represent the entire region.



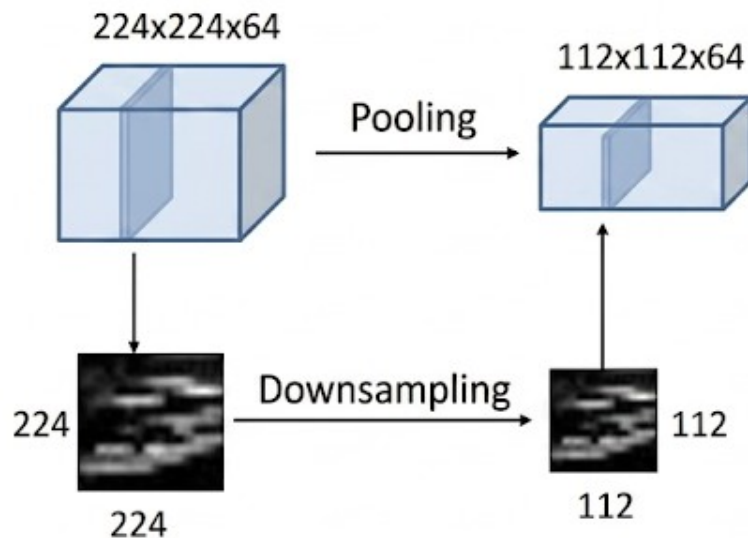
Pooling Operation Example

- Pooling operation is performed independently on each feature response map;
- *Pooling on a region of a feature response map means selecting a value in that region to represent the entire region.*



Pooling Operation Example

- Pooling operation is performed independently on each feature map;
- Pooling a specific region of a feature map means specifying a value to represent the whole region.



Single Feature Map

5	2	1	4
6	3	3	8
2	3	4	2
1	1	2	3

Max Pooling

Pooling Window: 2×2
Pooling Stride: 2

Max-Pooled Feature Map

6	8
3	4

After the operation, 75% of the activation information in the feature map is lost, but the number of feature maps remains unchanged.

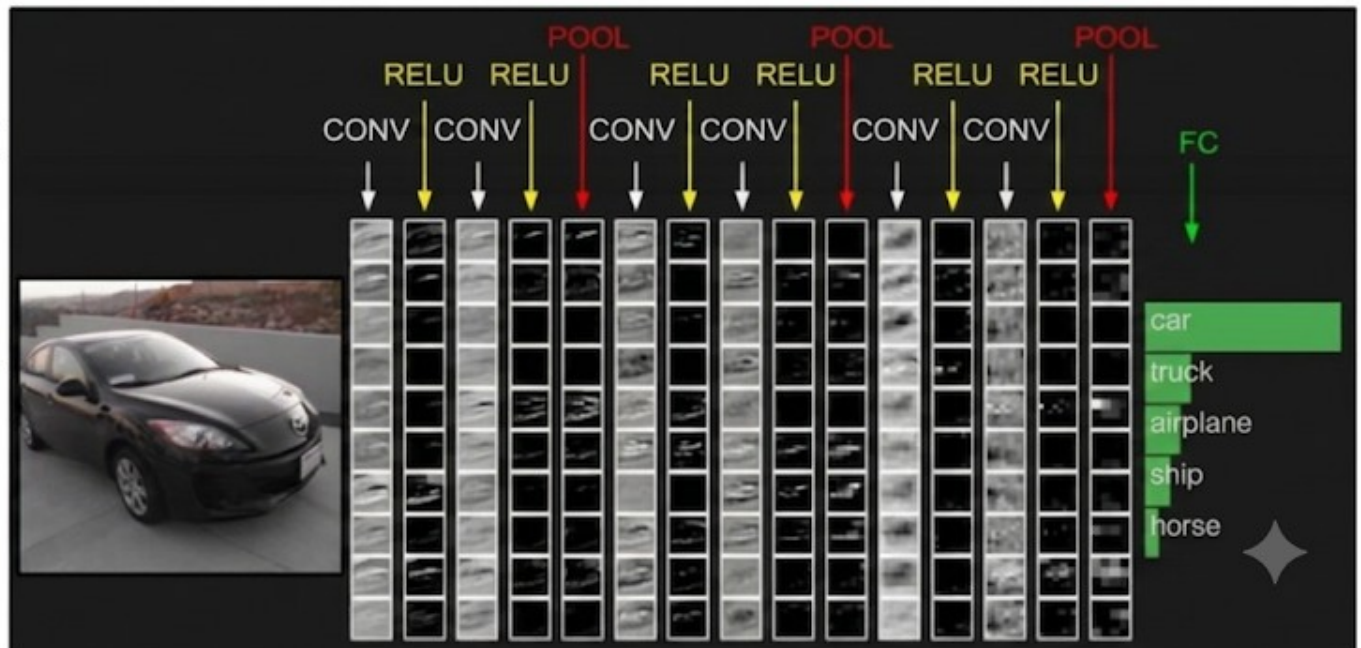
Convolutional Neural Network

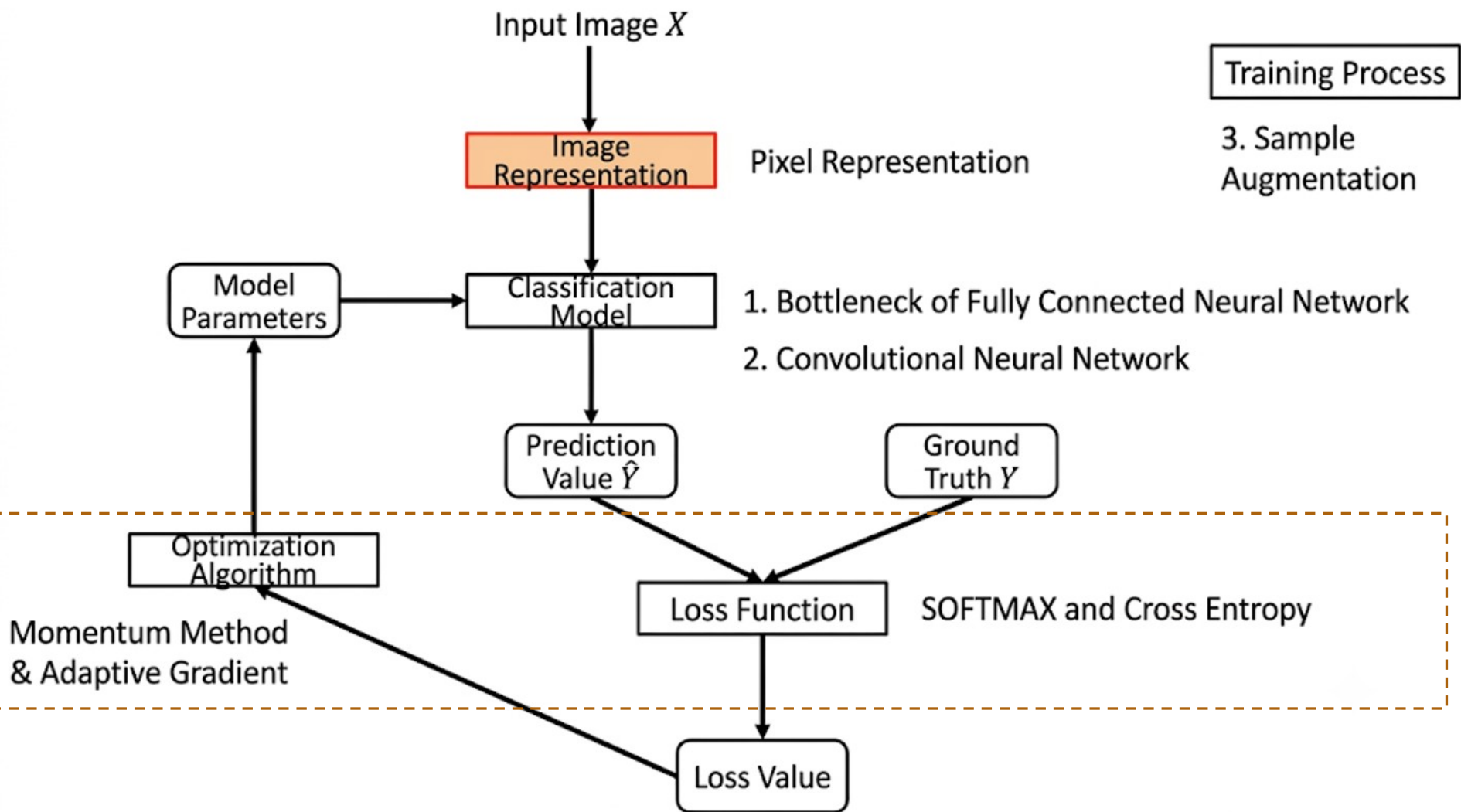
CONV — Convolutional Layer

RELU — Activation Layer

POOL — Pooling Layer

FC — Fully Connected Layer





Loss Function & Optimization Algorithms

- Loss Function: Cross Entropy Loss
- Optimization Algorithms: SGD, SGD with Momentum, and ADAM

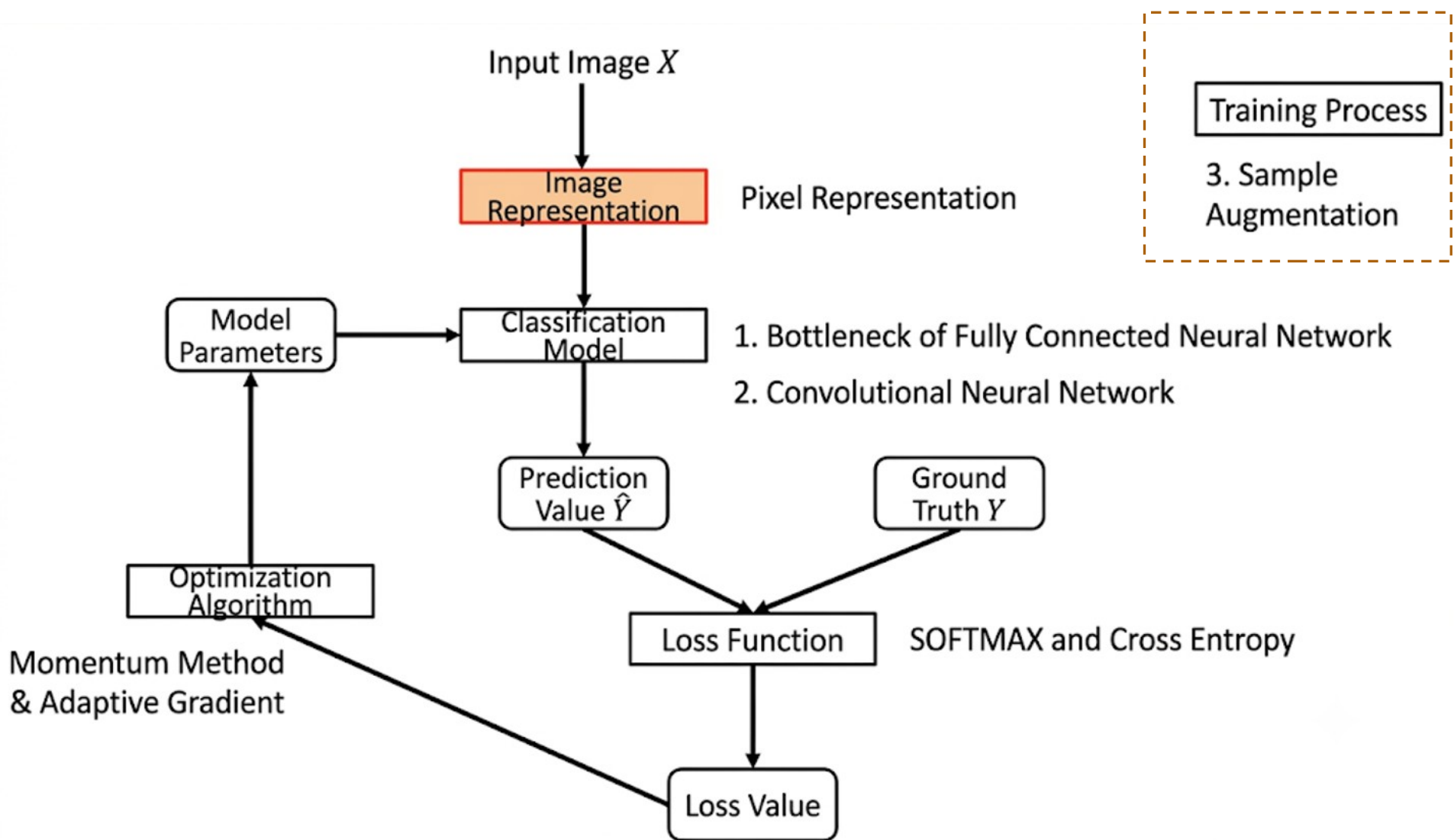
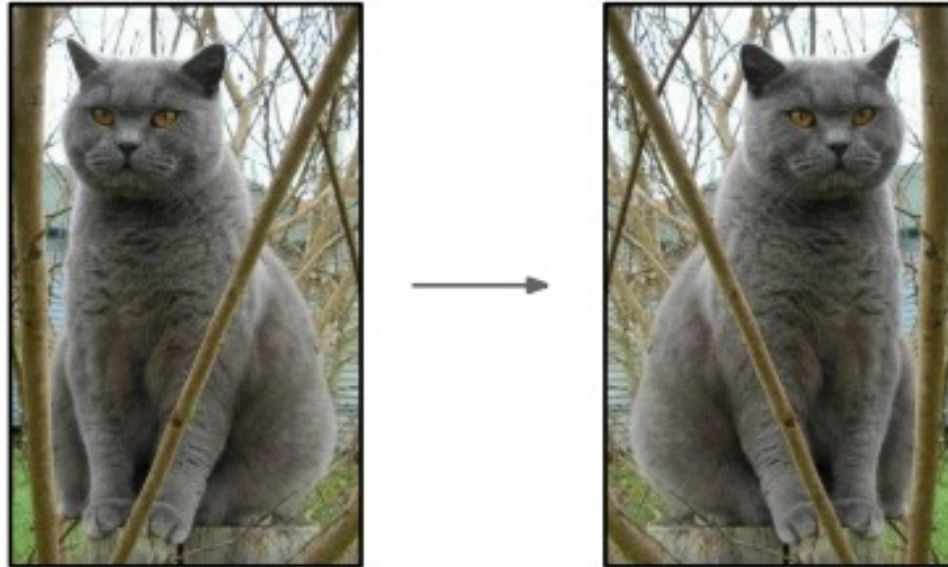


Image Enhancement

- **Problem:** Overfitting occurs when too few training samples exist, leading to an inability to train a model that generalizes well to new data.
- **Data Enhancement:** Data enhancement is the process of generating more training data from existing samples by applying various random transformations to generate plausible images and increase the number of samples.
- **Goal of Data Enhancement:** During training, the model does not look at the **exact same image twice**. This allows the model to observe more aspects of the data, thus improving its generalization capability.

Data Augmentation Flipping



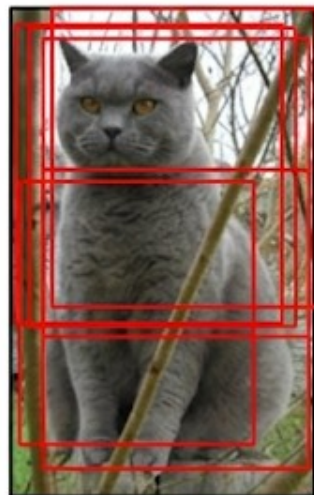
Data Augmentation — Random Cropping & Scaling

Taking the data augmentation method in Residual Networks as an example

Input Requirement: 224x224 color image

Training Stage: Randomly crop at different scales and locations

1. Randomly choose a scale L between $[256, 480]$
2. Scale the training sample to short edge = L
3. Randomly sample a 224×224 region from that sample



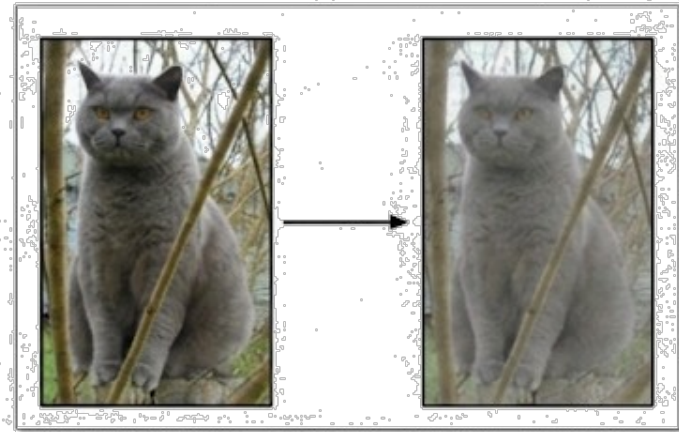
Testing Stage: Crop according to a set of predefined methods

1. Scale the image to 5 sizes: $\{224, 256, 384, 480, 640\}$
2. For each scaled image and its mirror image, crop a 224×224 region at four corners and the center to obtain 10 images

Data Augmentation — — Color Jittering

Operation Steps:

1. Use Principal Component Analysis (PCA) to extract the principal axes of the color data ([RGB]) of the current image;
2. Randomly sample an offset along the direction of the principal axes;
3. Add the offset to each pixel of the current image.



Sample Augmentation — Other Schemes

Randomly combine the following operations:

- Translation

- Rotation

- Stretching

- Radial Distortion

(Refer to the Camera Geometry chapter for related descriptions)

- Cropping

...